

Table of Contents

Early Claim and Lapsed Prediction	2
1. Business Understanding	2
2. Data Understanding	2
2.1 Features Breakdown	2
2.1.1 MiCare Claim	2
2.1.2 Traditional	6
2.1.3 Investment Linked	10
2.1.4 Customer	14
2.2 Merge Datasets	14
2.2.1 Dataset of Year 2018	15
2.2.2 Dataset of Year 2019 - 2021	16
2.2.3 Combined Dataset Year 2018 and 2019 - 2021	17
2.3 Read Data	17
2.4 Exploratory Data Analysis (EDA)	27
3. Data Preparation	38
3.1 Selecting Data	38
3.2 Data Cleaning	38
3.2.1 Remove Missing Values	38
3.2.2 Remove Constant & Low Variance Features	39
3.2.3 Remove Outliers	41
3.2.4 Handling High Cardinality Features	45
3.2.5 Remove Highly Correlated Features	48
3.3 Data Transformation (Feature Engineering & Feature Scaling / Encoding)	58
3.3.1 Feature Encoding (Categorical Data)	58
3.4 Handle Imbalanced Dataset	58
4. Modeling	59
4.1 LightGBM	59
4.2 LightGBM (Overall Metrics)	64
4.2.1 Accuracy, Precision, Recall, F1 and AUC Scores	64
4.2.2 LightGBM (Confusion Matrix)	66
4.2.3 LightGBM (ROC AUC Curve)	68
4.2.4 LightGBM (Feature Importance)	69
4.2.5 LightGBM (Interpretation)	69
4.2.6 LightGBM (Save Model)	69
4.2.7 LightGBM (Load Model)	70

Early Claim and Lapsed Prediction

1. Business Understanding

Tokio Marine Life Insurance Malaysia (TMLM) was looking for a way to help them automate the process of identifying which insurance claims are early claimed. Previously, this process was done manually by humans, which resulted in slow processing speed and erroneous indirectly as erroneous increases when people were exhausted. So, TMLM has collaborated with Tunku Abdul Rahman University (TARUC) as well as Dynafront company to involve in this project to build a model that can help them to classify which insurance claims are early claimed and which are not so that the process of identifying which insurance claims are early claimed can be boosted which can lead to higher productivity. Throughout this project, TARUC will provide domain knowledge of data science, machine learning and Artificial Intelligence, while TMLM will provide us with data and insurance claims knowledge.

2. Data Understanding

We have four datasets:

- **MiCare Claim:** Medical claims from policyholders in 2018.
- **RAT501_1(Trad):** Details of Traditional Life Insurance.
- **RAT501_2(IL):** Details of Investment-Linked.
- **RAT501_Customer_Extraction:** Customer Details.

2.1 Features Breakdown

2.1.1 MiCare Claim

Variable Name	Description
Year	Year of claim
Month	Month of claim
POLICY_NO	Policy Number
ClaimType	Claim type (Cashless/Reimbursement)

DOA	Date of Admission
DOD	Date of Discharge
RBRCExpPay	Daily Room & Board (Room Expenses)
RBFMExpPay	Daily Room & Board (Food/Meals/GST Expenses)
ICURCExpPay	ICU Room Expenses
SASurgeonExpPay	Surgeon Expenses
SAAnaExpPay	Anesthetic Expenses
OperatingExpPay	Operating Theater Expenses
HSSExpPayTotal	Total Expenses of Hospital Services & Supplies
HABNCExpPay	Nursing Care / Procedure / Treatment Expenses
HABMedExpPay	Medicine / Pharmacy / Injection / HomeDrugs Expenses
HABLabExpPay	Laboratory / Diagnostic / X-ray / Scan / Imaging Expenses
HSSTherapyExpPay	Therapy / Physiotherapy Expenses
HSSExpPay	Medical Supplies / Consumables Expenses
HSSOtherExpPay	Hospital Services & Supplies / Others Expenses
PhysicianExpPay	In-Hospital Physician Visit Expenses
LodgerExpPay	Lodger Expenses

AMBExpPay	Ambulance Expenses
MedRptExpPay	Medical Report Expenses
DSSurgExpPay	Surgical Expenses
DSHCEXPAY	Hospital Expenses
PABDiagExpPay	Pre-Hospital Diagnostic Tests Expenses
PABSCFSEXPAY	Pre-Hospital Specialist Consultation Expenses
PHTEXPAY	Post Hospitalization Treatment
AccOPEXPAY	Emergency Accidental Outpatient Treatment Expenses
AccDenExpPay	Emergency Accidental Dental Treatment Expenses
PABSCFSecExpPay	Second Surgical Opinion Expenses
MajorExpPay	Major Surgical Bonus Expenses
OPPhyExpPay	Outpatient Physiotherapy Treatment Expenses
MthOPCancerExpPay	Monthly Out-patient Kidney Cancer Treatment Expenses
OPCancerExpPay	Annual Out-patient Kidney Cancer Treatment Expenses
MthOPKidExpPay	Monthly Out-patient Kidney Dialysis Treatment Expenses
LifeOPKidExpPay	Annual Out-patient Kidney Dialysis Treatment Expenses
OrganExpPay	Organ Transplantation Dialysis Expenses

IPPhyExpPay	In-Patient Treatment for Mental Illness Expenses
PhysioExpPay	In-Hospitalization Physiotherapy Treatment Expenses
ImplantExpPay	Surgical Implant Expenses
DailyCashExpPay	Daily Cash Allowance at Government Hospital
ESTExpPay	Emergency Sickness Treatment Expenses
HNCTIExpPay	Home Nursing Care Expenses due to Terminal Illness
MaternityExpPay	Maternity Benefit
DeaBenExpPay	Death Benefit
AccDeaBenExpPay	Accidental Benefit
FuneExpPay	Funeral Expenses
DreadExpPay	Dread Disease Benefit / Max Limit for GH
PerDisExpPay	Total and Permanent Disability Expenses
TotalCopay	Total Co-Payment (Total medical expenses that customers should bear as their insurance policy doesn't fully bear the costs)
RBStay	Days of Room and Boards Stay
ICUStay	Days of ICU Stay
Gender	Sex
InceptionDate	Inception Date (Insurans effective start date)

Age	Age
HospitalCode	Hospital Code (Each different code represents a hospital)
DiagCode	Diagnosis Code (ICD code)
DiagDesc	Diagnosis Description
ClaimNo	Claims Number
ClaimIssuedDT	Claims Issued Date
DoctorCode	Doctor Code (Each different code represents a doctor)
Incured Amount	Incurred Amount
InsuredAmt	Insured Amount
excess paid by pt	Formula: Incured Amount - Insured Amount May include the cost of co-payment as well as some medical costs that the insurance policy doesn't bear
DiagProcedure	Diagnosis Procedure (details)
Type	Ordinary Life (Traditional) / Investment-linked

2.1.2 Traditional

Variable Name	Description
POLICY_NO	Policy number
BRANCH_CODE	Branch code
POLICY_STATUS	Policy status

CURRENCY_CODE	Currency code
RISK_CODE	Component code
LIFE_COVERAGE	Life Coverage
COVER_NO	Number of Coverage
RIDER_NO	Number of Rider
RISK_TERM	Term of basic plan
RISK_STATUS	Risk status
POL_ISSUE_YEAR	Year of policy issue date
POL_COMMENCE_DATE	Policy commencement date (CCan treat as policy start date)
SEX	Gender of the first life insured
OCCUPATION_CLASS	Occupation class (A, B, C, D)
RACE	Race code of the first life assured
MARITAL_STATUS	Marital status of the first life assured
ENTRY_AGE	The age at which the customer owned the policy
SMOKER	Smoker / Non-Smoker status
PAYMENT_MODE	Code of Payment Mode
RSK_SUM_ASSURE	Risk Sum Assured It is the Amount of Risk that the company takes on

	your life. It is the amount assured on your death (Sum Assured). A sum assured is a fixed amount that is paid to the policy holder in the unfortunate event of the policyholder's demise. The insurance company pays this money as per the sum chosen by you at the time of purchasing the policy.
ANUAL_PREMIUM	Annualized premium (exclude extra premium)
ANUAL_EX_PREMIUM	Annualized extra premium
MATURE_AGE	Risk cessation age / maturity age of the policy
PAYMNT_TERM	Payment term of basic plan How long (in years) the customer wants to pay for his/her policy
SUBSTD_FLAG	Substandard flag of the first life assured
EXP_DATE	Expiry Date/ Risk Cessation Date
RSK_COMMENCE_DATE	Risk commencement date
RSK_STATUS_DATE	Last date of risk status changed
PAR_INDICATE	Par / Non-Par Indicator
POL_LAPSE_DATE	Policy lapse date either from history or potential lapse date
POL_STATUS_DATE	Last date of policy status changed
POLICY_YEAR	Year of policy
EXTRA_LOAD	Extra Loading in percentage (%) It is an amount of additional loading that Tokio Marine

	imposes on the customers for the higher risk customers like customers who have high blood pressure, pre-existing conditions like diabetes, etc.
POL_HOLDER_TYPE	Channel
SERVICE_AGENT_NO	Servicing Agent number
SERVICE_AGENT_SEX	Servicing agent gender
SERVICE_AGENT_RACE	Servicing agent race
SERVICE_AGENT_BRANCH_CODE	Servicing agent branch code
SERVICE_AGENT_EDU_LEVEL	Servicing agent educational level
SERVICE_AGENT_AGE	Servicing agent age
SERVICE_AGENT_JOIN_DATE	Servicing agent join date
SERVICE_AGENT_OCCUPATION_CODE	Servicing agent occupation code
SERVICE_AGENT_POSTCODE	Servicing agent postcode
RISK_STATUS	Risk status
PREM_STATUS	Premium status
LAST_PAYMT_DATE	Last payment date
FIRST_ISSUE_DATE	First issue date
POL_ISSUE_DATE	Policy issue date
PAID_TO_DATE	Date of last premium payment

CLAIM_RECOD_DATE	Claims Coverage Start Date
PAYMENT_METHOD	Payment method
SELL_AGENT_NO	Selling agent number
SELL_AGENT_SEX	Selling agent gender
SELL_AGENT_RACE	Selling agent race
SELL_AGENT_EDU_LEVEL	Sell agent educational level
SELL_AGENT_AGE	Sell agent age
SELL_AGENT_JOIN_DATE	Sell agent join date
SELL_AGENT_OCCUPATION_CODE	Sell agent occupation code
SELL_AGENT_POSTCODE	Sell agent postcode
SELL_AGENT_BRANCH_CODE	Selling agent branch code (Original agent branch code)
FLAT_RATE_LOAD	Flat Rate Loading
FLAT_RATE_LOAD_YEAR	Flat rate loading year

2.1.3 Investment Linked

Variable Name	Description
POLICY_NO	Policy number
BRANCH_CODE	Branch code
POLICY_STATUS	Policy status

CURRENCY_CODE	Currency code
RISK_CODE	Component code
LIFE_COVERAGE	Life Coverage
COVER_NO	Number of Coverage
RIDER_NO	Number of Rider
RISK_TERM	Term of basic plan
RISK_STATUS	Risk status
POL_ISSUE_YEAR	Year of policy issue date
POL_COMMENCE_DATE	Policy commencement date
SEX	Gender of the first life insured
OCCUPATION_CLASS	Occupation class (A, B, C, D)
RACE	Race code of the first life assured
MARITAL_STATUS	Marital status of the first life assured
ENTRY_AGE	Entry age for component code
SMOKER	Smoker / Non-Smoker status
PAYMENT_MODE	Code of Payment Mode
RSK_SUM_ASSURE	Risk Sum Assured
SINGLE_PREMIUM	Single premium

MATURE_AGE	Risk cessation age / maturity age
PAYMNT_TERM	Payment term of basic plan
SUBSTD_FLAG	Substandard flag of the first life assured
EXP_DATE	Expiry Date/ Risk Cessation Date
RSK_COMMENCE_DATE	Risk commencement date
RSK_STATUS_DATE	Last date of risk status changed
POL_LAPSE_DATE	Policy lapse date either from history or potential lapse date
POL_STATUS_DATE	Last date of policy status changed
POLICY_YEAR	Year of policy
TOPUP_PREMIUM	Top up Premium
EXCESS_PREMIUM	Excess Premium
EXTRA_LOAD	Extra Loading in percentage (%) It is an amount of additional loading that Tokio Marine imposes on the customers for the higher risk customers like customers who have high blood pressure, pre-existing conditions like diabetes, etc.
SERV_BRANCH_CODE	Servicing agent branch code
POL HOLDER_TYPE	Channel
SERVICE_AGENT_NO	Servicing Agent number
SERVICE_AGENT_BRANCH_CODE	Agents branch code

SERVICE_AGENT_SEX	Servicing agent gender
SERVICE_AGENT_RACE	Servicing agent race
SERVICE_AGENT_EDU_LEVEL	Servicing agent educational level
SERVICE_AGENT_AGE	Servicing agent age
SERVICE_AGENT_JOIN_DATE	Servicing agent join date
SERVICE_AGENT_OCCUPATION_CODE	Servicing agent occupation code
SERVICE_AGENT_POSTCODE	Servicing agent postcode
RISK_STATUS	Risk status
PREM_STATUS	Premium status
LAST_PAYMT_DATE	Last payment date
POL_ISSUE_DATE	Policy issue date
INDCATE_FLEX_PREM	Indicator for Flex Premium
PAID_TO_DATE	Date of last premium payment
RSK_COMMENCE_DATE	Risk commencement date
PAYMENT_METHOD	Payment method
SELL_AGENT_NO	Selling agent number
SELL_AGENT_BRANCH_CODE	Selling agent branch code (Original agent branch code)
SELL_AGENT_SEX	Selling agent gender

SELL_AGENT_RACE	Selling agent race
SELL_AGENT_EDU_LEVEL	Sell agent educational level
SELL_AGENT_AGE	Sell agent age
SELL_AGENT_JOIN_DATE	Sell agent join date
SELL_AGENT_OCCUPATION_CODE	Sell agent occupation code
SELL_AGENT_POSTCODE	Sell agent postcode
FACULATIVE_IND	Facultative Indicator

2.1.4 Customer

Variable Name	Description
POLICY_NO	Policy number
CLIENT_NO	Client number
POSTCODE	Postcode
STATE	State
WEIGHT	Weight
HEIGHT	Height
BMI	Body Mass Index

2.2 Merge Datasets

2.2.1 Dataset of Year 2018

```
# Read MiCare dataset
df_micare = pd.read_excel(r'MiCare Claim.xlsx')
```

```
# Read Trad dataset
df_trad =
pd.read_excel(r'\\10.188.78.123\CP_Shared\2018\RAT501_1(Trad)_v2.xlsx')
```

```
# Read IL dataset
df_IL = pd.read_excel(r'\\10.188.78.123\CP_Shared\2018\RAT501_2(IL)_v2.xlsx')
```

```
# Read customer dataset
df_cust = pd.read_excel(r'Customer data_2018_v2.xlsx')
```

```
df_trad['Type'] = ['OL'] * df_trad.shape[0]
df_IL['Type'] = ['IL'] * df_IL.shape[0]
```

```
policy_df = pd.concat([df_trad, df_IL], axis=0) # Concatenate Trad and IL
first
policy_df.sort_values(by=['POLICY_NO'], inplace=True)
policy_df.reset_index(inplace=True, drop=True)
```

```
# Drop again the print(duplicate rows and keep the last appearance
policy_df.drop_duplicates(keep='last', inplace = True)
print(policy_df.shape)
```

```
claim_df = policy_df.merge(df_cust[['POLICY_NO', 'STATE', 'BMI']],
on='POLICY_NO', how='left')# Then merge with Customer
claim_df = claim_df.merge(df_micare[['POLICY_NO', 'DOA',
'EarlyClaimAndLapsed']], on='POLICY_NO', how='left') # Then merge with MiCare
claim_df.sort_values(by=['POLICY_NO'], inplace=True)
claim_df.reset_index(inplace=True, drop=True)
claim_df.head(10)
```

```
claim_df.to_csv('2018_dataset.csv', index=False)
```

2.2.2 Dataset of Year 2019 - 2021

```
# Read MiCare dataset
df_micare1 = pd.read_excel(r'MiCare Claim 2019-2021.xlsx')
```

```
# Read Trad dataset
df_trad1 = pd.read_excel(r'2020-2021/trad_2020-2021.xlsx')
```

```
# Read IL dataset
df_IL1 = pd.read_excel(r'2020-2021/il_2020-2021.xlsx')
```

```
# Read customer dataset
df_cust1 = pd.read_excel(r'Customer Data_2019-2021_masked.xlsx')
```

```
df_trad1['Type'] = ['OL'] * df_trad1.shape[0]
df_IL1['Type'] = ['IL'] * df_IL1.shape[0]
```

```
policy_df = pd.concat([df_trad1, df_IL1], axis=0) # Concatenate Trad and IL
first
policy_df.sort_values(by=['POLICY_NO'], inplace=True)
policy_df.reset_index(inplace=True, drop=True)
```

```
policy_df1.drop_duplicates(keep='last', inplace = True)
```

```
claim_df1 = policy_df1.merge(df_cust1[['POLICY_NO', 'STATE', 'BMI']],
on='POLICY_NO', how='left')# Then merge with Customer
claim_df1 = claim_df1.merge(df_micare1[['POLICY_NO', 'DOA',
'EarlyClaimAndLapsed']], on='POLICY_NO', how='left') # Then merge with MiCare
claim_df1.sort_values(by=['POLICY_NO'], inplace=True)
claim_df1.reset_index(inplace=True, drop=True)
claim_df1.head(10)
```

```
claim_df1.to_csv('2019-2021_dataset.csv', index=False)
```

2.2.3 Combined Dataset Year 2018 and 2019 - 2021

```
old = pd.read_csv(r'2018_dataset.csv')
new = pd.read_csv(r'2019-2021_dataset.csv')

combined = pd.concat([old, new], axis=0)
combined.to_csv('2018-2019_dataset_new.csv', index=False)
```

2.3 Read Data

```
# Read Merged dataset
claim_df = pd.read_excel(r'2018-2021_dataset_new.xlsx')
claim_df
```

```
# Check whether there are whitespace value in the dataframe
claim_df.iloc[:, 32].values[0].isspace()
```

True

```
for col in range(claim_df.shape[1]):
    if claim_df.iloc[:, col].dtype == 'object':
        for index, val in enumerate(claim_df.iloc[:, col].values):
            claim_df.iloc[:, col].values[index] = str(val).strip()
            if str(val).isspace() or val is None or str(val).lower() == 'nan'
or str(val).lower() == 'none' or val == '':
                claim_df.iloc[:, col].values[index] = np.NaN
```

```
# Check whether the value is whitespace again
try:
    claim_df.iloc[:, 32].values[0].isspace()
except AttributeError:
    print(f'The value now becomes {claim_df.iloc[:, 32].values[0]}. Is it not
whitespace anymore')

import math
```

```
print(f"Check if it is np.NaN: {math.isnan(claim_df.iloc[:, 32].values[0])}")
```

The value now becomes nan. Is it not whitespace anymore
Check if it is np.NaN: True

```
# Columns that have mixed type problem
display(Markdown(f"#### Columns that have mixed type problem:"))
for index in (28, 32, 40, 46, 49, 57, 59, 63, 64, 65, 67):
    print(f"Column Index: {index}\nColumn Name: {claim_df.iloc[:,
index].name}\n")
```

Columns that have mixed type problem:

Column Index: 28

Column Name: PAR_INDICATE

Column Index: 32

Column Name: EXTRA_LOAD

Column Index: 40

Column Name: SERVICE_AGENT_JOIN_DATE

Column Index: 46

Column Name: FIRST_ISSUE_DATE

Column Index: 49

Column Name: CLAIM_RECOD_DATE

Column Index: 57

Column Name: SELL_AGENT_JOIN_DATE

Column Index: 59

Column Name: SELL_AGENT_POSTCODE

Column Index: 63

Column Name: SINGLE_PREMIUM

Column Index: 64

Column Name: TOPUP_PREMIUM

Column Index: 65

Column Name: EXCESS_PREMIUM

Column Index: 67

Column Name: FACULATIVE_IND

```
# Make sure these 2 columns are in float data type
claim_df['SELL_AGENT_POSTCODE'] = claim_df['SELL_AGENT_POSTCODE'].astype(float)
claim_df['SERVICE_AGENT_POSTCODE'] =
claim_df['SERVICE_AGENT_POSTCODE'].astype(float)
```

```
# Convert those columns that have mixed types of data
claim_df.iloc[:, 28] = claim_df.iloc[:, 28].apply(str)
claim_df.iloc[:, 32] = claim_df.iloc[:, 32].astype(float)
claim_df.iloc[:, 63] = claim_df.iloc[:, 63].astype(float)
claim_df.iloc[:, 64] = claim_df.iloc[:, 64].astype(float)
claim_df.iloc[:, 65] = claim_df.iloc[:, 65].astype(float)
claim_df.iloc[:, 67] = claim_df.iloc[:, 67].apply(str)

for x in claim_df.columns.tolist():
    if 'nan' in claim_df[x].value_counts().keys() or 'none' in
claim_df[x].value_counts().keys():
        for index, val in enumerate(claim_df[x].values):
            if val == 'nan' or val == 'none':
                claim_df[x].values[index] = np.NaN
```

```
print(f"\nInsurance claims dataset shape")
print("-----")
print("Rows      : " , claim_df.shape[0])
print("Columns   : " , claim_df.shape[1])
print("Shape      : " , claim_df.shape)
print("Features  : \n" , claim_df.columns.tolist())
```

Output:

Insurance claims dataset shape

```
-----
Rows      : 195504
Columns   : 72
Shape      : (195504, 72)
Features  :
['POLICY_NO', 'BRANCH_CODE', 'POLICY_STATUS', 'CURRENCY_CODE', 'RISK_CODE',
'LIFE_COVERAGE', 'COVER_NO', 'RIDER_NO', 'RISK_TERM', 'RISK_STATUS',
'POL_ISSUE_YEAR', 'POL_COMMENCE_DATE', 'SEX', 'OCCUPATION_CLASS', 'RACE',
```

```
'MARITAL_STATUS', 'ENTRY_AGE', 'SMOKER', 'PAYMENT_MODE', 'RSK_SUM_ASSURE',
'ANUAL_PREMIUM', 'ANUAL_EX_PREMIUM', 'MATURE_AGE', 'PAYMNT_TERM', 'SUBSTD_FLAG',
'EXP_DATE', 'RSK_COMMENCE_DATE', 'RSK_STATUS_DATE', 'PAR_INDICATE',
'POL_LAPSE_DATE', 'POL_STATUS_DATE', 'POLICY_YEAR', 'EXTRA_LOAD',
'POL HOLDER_TYPE', 'SERVICE_AGENT_NO', 'SERVICE_AGENT_SEX', 'SERVICE_AGENT_RACE',
'SERVICE_AGENT_BRANCH_CODE', 'SERVICE_AGENT_EDU_LEVEL', 'SERVICE_AGENT_AGE',
'SERVICE_AGENT_JOIN_DATE', 'SERVICE_AGENT_OCCUPATION_CODE',
'SERVICE_AGENT_POSTCODE', 'RISK_STATUS.1', 'PREM_STATUS', 'LAST_PAYMT_DATE',
'FIRST_ISSUE_DATE', 'POL_ISSUE_DATE', 'PAID_TO_DATE', 'CLAIM_RECOD_DATE',
'PAYMENT_METHOD', 'SELL_AGENT_NO', 'SELL_AGENT_BRANCH_CODE', 'SELL_AGENT_SEX',
'SELL_AGENT_RACE', 'SELL_AGENT_EDU_LEVEL', 'SELL_AGENT_AGE',
'SELL_AGENT_JOIN_DATE', 'SELL_AGENT_OCCUPATION_CODE', 'SELL_AGENT_POSTCODE',
'FLAT_RATE_LOAD', 'FLAT_RATE_LOAD_YEAR', 'Type', 'SINGLE_PREMIUM',
'TOPUP_PREMIUM', 'EXCESS_PREMIUM', 'INDCATE_FLEX_PREM', 'FACULATIVE_IND',
'STATE', 'BMI', 'DOA', 'EarlyClaimAndLapsed']
```

```
# Since we load the data from CSV file, pandas will treat date columns as
string/object, so we are required convert them back to date
claim_df['DOA'] = pd.to_datetime(claim_df['DOA'], dayfirst=True)
claim_df['POL_COMMENCE_DATE'] = pd.to_datetime(claim_df['POL_COMMENCE_DATE'],
dayfirst=True)

# View the info after changing to datetime64[ns] data type
claim_df[['DOA', 'POL_COMMENCE_DATE']].info()
```

Output:

```
RangeIndex: 195504 entries, 0 to 195503
Data columns (total 2 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   DOA                   195504 non-null  datetime64[ns]
 1   POL_COMMENCE_DATE     195504 non-null  datetime64[ns]
dtypes: datetime64[ns](2)
memory usage: 3.0 MB
```

```
# View some of the samples above after done converting them to date
claim_df[['DOA', 'POL_COMMENCE_DATE']].head()
```

```
# Gap between Policy Commencement Date and DOA.
claim_df['GAP_BET_DOA_PCD'] = claim_df.apply(lambda row: (row['DOA'] -
row['POL_COMMENCE_DATE']).days, axis=1)
```

```

# Calculate whether the days in GAP_BET_DOA_PCD is within 2 years, replace with
1 if yes, otherwise 0
# 1 indicating within 2 years
# 0 indicating not within 2 years
claim_df['GAP_BET_DOA_PCD'] = claim_df['GAP_BET_DOA_PCD'].apply(lambda x: 1 if
x <= 730 else 0)
claim_df.rename(columns = {'GAP_BET_DOA_PCD': 'IS_POL_IN_2YEARS'}, inplace=True)
claim_df['IS_POL_IN_2YEARS'] = claim_df['IS_POL_IN_2YEARS'].apply(str)

# View some of the samples above after done converting them to date
claim_df[['IS_POL_IN_2YEARS']]

```

Select those policy data that is within 2 years timeframe

```

claim_df = claim_df.loc[claim_df['IS_POL_IN_2YEARS'] == '1']
claim_df['IS_POL_IN_2YEARS'].value_counts()

```

Output:

```

1      53987
Name: IS_POL_IN_2YEARS, dtype: int64

```

```

# Drop the 'IS_POL_IN_2YEARS' column bcus it is not useful anymore
claim_df.drop(labels=['IS_POL_IN_2YEARS'], axis=1, inplace=True)

# Move the "EarlyClaimAndLapsed" column to the last
last_column = claim_df.pop('EarlyClaimAndLapsed')
claim_df.insert(claim_df.shape[1], 'EarlyClaimAndLapsed', last_column)

# Display data
claim_df

```

```

# Check if there any duplicate rows left
claim_df[claim_df.duplicated()].shape[0]

```

Output:

```

1      6962

```

```

year_date_cols = ['POL_ISSUE_YEAR', 'POL_COMMENCE_DATE', 'EXP_DATE', 'DOA',

```

```

'LAST_PAYMT_DATE',
        'RSK_COMMENCE_DATE', 'RSK_STATUS_DATE', 'POL_LAPSE_DATE',
'POL_STATUS_DATE', 'FIRST_ISSUE_DATE',
        'PAID_TO_DATE', 'CLAIM_RECOD_DATE', 'FLAT_RATE_LOAD_YEAR',
'POL_ISSUE_DATE', 'POLICY_YEAR',
        'SERVICE_AGENT_JOIN_DATE', 'SELL_AGENT_JOIN_DATE']

agent_cols = ['SERVICE_AGENT_NO', 'SERVICE_AGENT_BRANCH_CODE', 'SELL_AGENT_NO',
'SELL_AGENT_BRANCH_CODE',
        'SELL_AGENT_RACE', 'SELL_AGENT_SEX', 'SERVICE_AGENT_RACE',
'SERVICE_AGENT_SEX']

payment_cols = []

```

```

'''
SINGLE_PREMIUM: 74260 of 74267 rows are blank. More than half (99.99%) of the
data are blank.
TOPUP_PREMIUM: 74260 of 74267 rows are blank. More than half (99.99%) of the
data are blank.
EXCESS_PREMIUM: All blank (100%).
INDCATE_FLEX_PREM: 71483 of 74267 rows are blank. More than half (96.25%) of
the data are blank.
ANUAL_PREMIUM: 46395 of 74267 rows are blank. More than half (62.47%) of the
data are blank.
ANUAL_EX_PREMIUM: 46395 of 74267 rows are blank. More than half (62.47%) of the
data are blank.
PREM_STATUS: No blank.
'''

prem_cols = ['SINGLE_PREMIUM', 'TOPUP_PREMIUM', 'EXCESS_PREMIUM',
'INDCATE_FLEX_PREM', 'ANUAL_PREMIUM', 'ANUAL_EX_PREMIUM',
        'PREM_STATUS']

import warnings
warnings.filterwarnings('ignore')
sweet_report = sv.analyze(source=claim_df[prem_cols + ['EarlyClaimAndLapsed']],
target_feat='EarlyClaimAndLapsed')
sweet_report.show_notebook(w="100%", h="Full", layout='vertical')

```

'''

CURRENCY_CODE: All rows only have 1 category value only which is 'RM'.

MATURE_AGE: Range from 13 to 90.

PAR_INDICATE: 46395 of 74267 rows are blank. More than half (62.47%) of the data are blank.

14117 of 74267 rows are 'P2' valued.

9781 of 74267 rows are 'P1' valued.

3974 of 74267 rows are 'NP' valued.

FACULATIVE_IND: 27872 of 74267 rows are blank. More than half 1/4 (37.53%) of the data are blank.

Only 1 category value only which is 'N' (62.47%).

An indicator of whether the policy is under reinsurance facultative agreement. Advised to drop this field

FLAT_RATE_LOAD: 46395 of 74267 rows are blank. More than half (62.47%) of the data are blank.

Only 1 category value only which is 0 (37.26%).

LIFE_COVERAGE: Range from 1 to 2

COVER_NO: Do not include bcus don't have 2019-2021 data. Advised to drop this field

RISK_TERM:

RIDER_NO: Do not include bcus don't have 2019-2021 data. Advised to drop this field

'''

```
other_cols = ['POLICY_NO', 'BRANCH_CODE', 'CURRENCY_CODE', 'MATURE_AGE',  
'PAR_INDICATE', 'FACULATIVE_IND',  
              'FLAT_RATE_LOAD', 'LIFE_COVERAGE', 'COVER_NO', 'RISK_TERM',  
'POLICY_STATUS', 'RIDER_NO',  
              'RISK_STATUS', 'RISK_STATUS.1'] # 'BRANCH_CODE', 'SEX', 'RACE'
```

```
sweet_report = sv.analyze(source=claim_df[other_cols +
['EarlyClaimAndLapsed']], target_feat='EarlyClaimAndLapsed')
sweet_report.show_notebook(w="100%", h="Full", layout='vertical')
```

```
total_drop_cols = len(year_date_cols + agent_cols + payment_cols + other_cols +
prem_cols)
print(f'Total Columns to be Dropped: {total_drop_cols} out of
{claim_df.shape[1]}')
print(f'Total Columns Left: {claim_df.shape[1] - total_drop_cols}')

# Drop useless columns
claim_df.drop(columns=year_date_cols + agent_cols + payment_cols + other_cols +
prem_cols, inplace=True, axis=1)
claim_df.drop_duplicates(inplace=True)
```

Output:

Total Columns to be Dropped: 46 out of 72
Total Columns Left: 26

```
# Check again if there any duplicate rows left
claim_df[claim_df.duplicated()].shape[0]
```

Output:

0

```
claim_df.dropna(subset=['SELL_AGENT_POSTCODE', 'SERVICE_AGENT_POSTCODE'],
inplace=True)
claim_df.reset_index(inplace=True, drop=True)
```

```
def mapPostCode(claim_df):
    i = 0
    while i < claim_df.count():
        if claim_df[i] in range(50000, 60001):
            claim_df[i] = 'KUALA LUMPUR'
        elif str(claim_df[i]).startswith('62'):
            claim_df[i] = 'PUTRAJAYA'
        elif str(claim_df[i]).startswith('87'):
```

```

        claim_df[i] = 'LABUAN'
    elif claim_df[i] in range (1000,2801) :
        claim_df[i] = 'PERLIS'
    elif claim_df[i] in range (5000,9811) or claim_df[i] == 14290 or
claim_df[i] == 14390 or claim_df[i] == 34950:
        claim_df[i] = 'KEDAH'
    elif claim_df[i] in range (10000,14401) :
        claim_df[i] = 'PENANG'
    elif claim_df[i] in range (15000,18501) :
        claim_df[i] = 'KELANTAN'
    elif claim_df[i] in range (20000,24301) :
        claim_df[i] = 'TERENGGANU'
    elif claim_df[i] in range (25000,28801) or
str(claim_df[i]).startswith('39') or str(claim_df[i]).startswith('49') or
claim_df[i] == 69000:
        claim_df[i] = 'PAHANG'
    elif claim_df[i] in range (30000,36811) :
        claim_df[i] = 'PERAK'
    elif claim_df[i] in range (40000,48301) or claim_df[i] in range
(63000,63301) or claim_df[i] == 64000 or claim_df[i] in range (68000,68101):
        claim_df[i] = 'SELANGOR'
    elif claim_df[i] in range (70000,73501) :
        claim_df[i] = 'NEGERI SEMBILAN'
    elif claim_df[i] in range (75000,78301) :
        claim_df[i] = 'MELAKA'
    elif claim_df[i] in range (79000,86901) :
        claim_df[i] = 'JOHOR'
    elif claim_df[i] in range (88000,91301) :
        claim_df[i] = 'SABAH'
    elif claim_df[i] in range (93000,98851) :
        claim_df[i] = 'SARAWAK'
    i+=1

mapPostCode(claim_df['SELL_AGENT_POSTCODE'])
mapPostCode(claim_df['SERVICE_AGENT_POSTCODE'])
claim_df

```

```

import numbers

claim_df.rename(columns = {'SELL_AGENT_POSTCODE': 'SELL_AGENT_STATE',
                           'SERVICE_AGENT_POSTCODE': 'SERVICE_AGENT_STATE'},
                inplace = True)

fail_con_count = {}
display(Markdown(f"#### Total number rows: {claim_df.shape[0]}"))
for agnt_postcode in ['SELL_AGENT_STATE', 'SERVICE_AGENT_STATE']:
    count = 0
    for x in claim_df[agnt_postcode].values:
        if isinstance(x, numbers.Number) == True:
            count = count + 1
    fail_con_count[agnt_postcode] = count
    print(f"Column Name: {agnt_postcode}")
    print(f"Number of postcodes that fail to convert to state: {count}")

```

Output:

Total number rows: 16439

Column Name: SELL_AGENT_STATE

Number of postcodes that fail to convert to state: 0

Column Name: SERVICE_AGENT_STATE

Number of postcodes that fail to convert to state: 0

```

if fail_con_count['SELL_AGENT_STATE'] != 0 or
fail_con_count['SERVICE_AGENT_STATE'] != 0:
    for col in ['SELL_AGENT_STATE', 'SERVICE_AGENT_STATE']:
        for poskod in claim_df[col]:
            if isinstance(poskod, numbers.Number) == True:
                print(f'Postcode {poskod} does not exist in Malaysia')
                claim_df = claim_df.loc[claim_df[col] != poskod]

```

```

display(Markdown(f"#### Total number rows: {claim_df.shape[0]}"))
for agnt_postcode in ['SELL_AGENT_STATE', 'SERVICE_AGENT_STATE']:
    count = 0
    for x in claim_df[agnt_postcode].values:
        if isinstance(x, numbers.Number) == True:
            count = count + 1

```

```
print(f"Column Name: {agnt_postcode}")
print(f"Number of postcodes that fail to convert to state: {count}")
```

Output:

Total number rows: 16439

Column Name: SELL_AGENT_STATE

Number of postcodes that fail to convert to state: 0

Column Name: SERVICE_AGENT_STATE

Number of postcodes that fail to convert to state: 0

```
for x in claim_df.columns.tolist():
    if 'none' in claim_df[x].value_counts().keys():
        print(x)
```

```
# Check again if there any duplicate rows left
print(f"Any duplicate rows left?\nAnswer:
{claim_df[claim_df.duplicated()].shape[0]}\n")

# If there any duplicate rows left, drop it
if claim_df[claim_df.duplicated()].shape[0] > 0:
    print("\nDuplicate rows will be cleared now...")
    claim_df.drop_duplicates(inplace=True)
    print(f"Any duplicate rows left?\nAnswer:
{claim_df[claim_df.duplicated()].shape[0]}")
```

Output:

Any duplicate rows left?

Answer: 1

Duplicate rows will be cleared now...

Any duplicate rows left?

Answer: 0

2.4 Exploratory Data Analysis (EDA)

```
print(f"\nMerged (claim_df) dataset shape")
print("-----")
print ("Rows      : " , claim_df.shape[0])
print ("Columns   : " , claim_df.shape[1])
print ("Shape      : " , claim_df.shape)
```

```
print ("Features : \n" , claim_df.columns.tolist())
```

Output:

Merged (claim_df) dataset shape

```
-----  
Rows      : 16438  
Columns   : 26  
Shape     : (16438, 26)  
Features  :  
['RISK_CODE', 'SEX', 'OCCUPATION_CLASS', 'RACE', 'MARITAL_STATUS', 'ENTRY_AGE',  
'SMOKER', 'PAYMENT_MODE', 'RSK_SUM_ASSURE', 'PAYMNT_TERM', 'SUBSTD_FLAG',  
'EXTRA_LOAD', 'POL HOLDER_TYPE', 'SERVICE_AGENT_EDU_LEVEL', 'SERVICE_AGENT_AGE',  
'SERVICE_AGENT_OCCUPATION_CODE', 'SERVICE_AGENT_STATE', 'PAYMENT_METHOD',  
'SELL_AGENT_EDU_LEVEL', 'SELL_AGENT_AGE', 'SELL_AGENT_OCCUPATION_CODE',  
'SELL_AGENT_STATE', 'Type', 'STATE', 'BMI', 'EarlyClaimAndLapsed']
```

```
claim_df.info()
```

Output:

Int64Index: 16438 entries, 0 to 16438

Data columns (total 26 columns):

#	Column	Non-Null Count	Dtype
0	RISK_CODE	16438 non-null	object
1	SEX	16438 non-null	object
2	OCCUPATION_CLASS	16438 non-null	object
3	RACE	16373 non-null	object
4	MARITAL_STATUS	16438 non-null	object
5	ENTRY_AGE	16438 non-null	int64
6	SMOKER	16438 non-null	object
7	PAYMENT_MODE	16438 non-null	int64
8	RSK_SUM_ASSURE	16438 non-null	int64
9	PAYMNT_TERM	16438 non-null	int64
10	SUBSTD_FLAG	6334 non-null	object
11	EXTRA_LOAD	13742 non-null	float64
12	POL HOLDER_TYPE	16438 non-null	object
13	SERVICE_AGENT_EDU_LEVEL	15233 non-null	object
14	SERVICE_AGENT_AGE	16194 non-null	float64
15	SERVICE_AGENT_OCCUPATION_CODE	16194 non-null	object
16	SERVICE_AGENT_STATE	16438 non-null	object
17	PAYMENT_METHOD	16438 non-null	object
18	SELL_AGENT_EDU_LEVEL	15523 non-null	object
19	SELL_AGENT_AGE	16390 non-null	float64
...			
24	BMI	16427 non-null	float64

```
25 EarlyClaimAndLapsed          16438 non-null object
dtypes: float64(4), int64(4), object(18)
memory usage: 3.4+ MB
```

```
# Reset the index of the dataframe so that the index values can be moved out as
the first column in the dataframe.
display(Markdown(f"#### Unique values of each categorical features:"))
pd.DataFrame(claim_df[claim_df.select_dtypes(include =
['object']).keys().tolist()].nunique(),
              columns=['Number of Unique Values']).reset_index().rename(columns
= {'index': 'Categorical Features'}).sort_values(by=['Number of Unique Values'],
ascending = False)
```

```
# Description of the dataset
claim_df.describe()
```

```
pie_display = claim_df

# labels
lab = pie_display["EarlyClaimAndLapsed"].value_counts().keys().tolist()

# values
val = claim_df["EarlyClaimAndLapsed"].value_counts().values.tolist()

trace = go.Pie(labels = lab , values = val, marker = dict(colors =
['blue' , 'green'], line = dict(color = "white", width = 1.3)), rotation = 90,
hoverinfo = "label + value + text", hole = 0.5)
layout = go.Layout(dict(title = "Distribution of Early & Lapsed Claimed in
Data", title_x = 0.5, plot_bgcolor = "rgb(243, 243, 243)", paper_bgcolor =
"rgb(243, 243, 243)",))

import plotly.io as pio
pio.renderers.default='notebook'

fig = go.Figure(data = [trace], layout = layout)
py.iplot(fig)
```

```
# Detect any missing value from dataset
count = claim_df.isnull().sum().sort_values(ascending = False)
percentage = ((claim_df.isnull().sum() / len(claim_df) *
100).sort_values(ascending = False))
data_missing = pd.concat([count, percentage], axis = 1, keys = ['Count',
'Percentage (%)'])
data_missing = data_missing.loc[data_missing['Count'] != 0]

display(Markdown(f"#### Count and Percentage of missing values for the
columns:"))
data_missing = data_missing.reset_index()
data_missing.rename(columns = {'index': 'Features'}, inplace = True)
data_missing
```

```
display(Markdown(f"#### Total missing values: {data_missing.Count.sum()}"))
```

Output:

Total missing values: 15610

```
# Convert the missing value in EXTRA_LOAD field with 1 and others value with 0
claim_df['EXTRA_LOAD'][:] = claim_df['EXTRA_LOAD'].apply(lambda x: 0 if
math.isnan(x)==True else (0 if x==0 else 1))
claim_df['EXTRA_LOAD'] = claim_df['EXTRA_LOAD'].astype(int).astype(str)
claim_df['EXTRA_LOAD'].value_counts()
```

Output:

0 12984

1 3454

Name: EXTRA_LOAD, dtype: int64

```
# Drop 'SUBSTD_FLAG' column
claim_df.drop(labels=['SUBSTD_FLAG'], inplace=True, axis=1)
```

```
# View the missing values again
count = claim_df.isnull().sum().sort_values(ascending = False)
percentage = ((claim_df.isnull().sum() / len(claim_df) *
100).sort_values(ascending = False))
```

```

data_missing = pd.concat([count, percentage], axis = 1, keys = ['Count',
'Percentage (%)'])
data_missing = data_missing.loc[data_missing['Count'] != 0]

display(Markdown(f"#### Count and Percentage of missing values for the
columns:"))
data_missing = data_missing.reset_index()
data_missing.rename(columns = {'index': 'Features'}, inplace = True)
data_missing

```

```

# View the total missing values again
display(Markdown(f"#### Total missing values: {data_missing.Count.sum()}"))

```

Output:

Total missing values: 2810

```

# Drop all rows containing NaN values
claim_df.dropna(inplace=True)

# Shape of dataset after dropping NaN values
print('Shape of dataset after dropping NaN values:')
claim_df.shape

```

Output:

Shape of dataset after dropping NaN values:
(15096, 25)

```

...
In the STATE column, there may be some areas that are not part of Malaysia, so
we need to delete these
areas and select the areas that are part of Malaysia.
...
claim_df = claim_df.loc[(claim_df['STATE'] == 'JOHOR') | (claim_df['STATE'] ==
'KEDAH') | (claim_df['STATE'] == 'KELANTAN')
| (claim_df['STATE'] == 'KUALA LUMPUR') |
(claim_df['STATE'] == 'LABUAN')
| (claim_df['STATE'] == 'MELAKA') | (claim_df['STATE']
== 'NEGERI SEMBILAN')

```

```

        | (claim_df['STATE'] == 'PAHANG') | (claim_df['STATE']
== 'PENANG') | (claim_df['STATE'] == 'PERAK')
        | (claim_df['STATE'] == 'PERLIS') | (claim_df['STATE']
== 'SABAH')
        | (claim_df['STATE'] == 'SARAWAK') | (claim_df['STATE']
== 'SELANGOR')
        | (claim_df['STATE'] == 'TERENGGANU') |
(claim_df['STATE'] == 'PUTRAJAYA')]

# Replace the proper values for PAYMENT_MODE and convert the data type from
float to object
'''1 - Annual, 2: Half-Yearly, 4: Quarterly, 12: Monthly'''
claim_df['PAYMENT_MODE'] = claim_df['PAYMENT_MODE'].replace({12.: 'Monthly',
1.: 'Yearly',
                                                    2.: 'Half-Yearly',
4.: 'Quarterly'}).astype(str)
'''

Convert the data type from float to object, because the values of
RSK_SUM_ASSURE are not represented to the price
but for the packages that the policyholders bought
'''
claim_df['RSK_SUM_ASSURE'] = claim_df['RSK_SUM_ASSURE'].astype(int).astype(str)

```

```
claim_df.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
```

```
Int64Index: 15085 entries, 0 to 16437
```

```
Data columns (total 25 columns):
```

#	Column	Non-Null Count	Dtype
0	RISK_CODE	15085 non-null	object
1	SEX	15085 non-null	object
2	OCCUPATION_CLASS	15085 non-null	object
3	RACE	15085 non-null	object
4	MARITAL_STATUS	15085 non-null	object
5	ENTRY_AGE	15085 non-null	int64
6	SMOKER	15085 non-null	object
7	PAYMENT_MODE	15085 non-null	object
8	RSK_SUM_ASSURE	15085 non-null	object

```
9   PAYMNT_TERM                15085 non-null int64
10  EXTRA_LOAD                 15085 non-null object
11  POL_HOLDER_TYPE            15085 non-null object
12  SERVICE_AGENT_EDU_LEVEL     15085 non-null object
13  SERVICE_AGENT_AGE           15085 non-null float64
14  SERVICE_AGENT_OCCUPATION_CODE 15085 non-null object
15  SERVICE_AGENT_STATE         15085 non-null object
16  PAYMENT_METHOD              15085 non-null object
17  SELL_AGENT_EDU_LEVEL        15085 non-null object
18  SELL_AGENT_AGE              15085 non-null float64
19  SELL_AGENT_OCCUPATION_CODE   15085 non-null object
20  SELL_AGENT_STATE            15085 non-null object
21  Type                        15085 non-null object
22  STATE                       15085 non-null object
23  BMI                         15085 non-null float64
24  EarlyClaimAndLapsed         15085 non-null object
dtypes: float64(3), int64(2), object(20)
memory usage: 3.0+ MB
```

```
display(Markdown(f"#### Total missing values:
{claim_df.isnull().sum().sum()}"))
```

Output:

Total missing values: 0

```
# Check duplicate rows again
if claim_df[claim_df.duplicated()].shape[0] > 0:
    print(f"There are {claim_df[claim_df.duplicated()].shape[0]} duplicated
rows.")
    print("\nDuplicate rows will be cleared now...")
    claim_df.drop_duplicates(inplace=True)
else:
    print("Congratz! No duplicate rows.")
```

Output:

There are 220 duplicated rows.

Duplicate rows will be cleared now...

```
from dataprep.eda import plot
plot(claim_df)
```

```
# Using sweetwiz to generate report
import warnings
warnings.filterwarnings('ignore')

sweet_report = sv.analyze(source=claim_df, target_feat='EarlyClaimAndLapsed')
sweet_report.show_notebook(w="100%", h="Full", layout='vertical')
```

```
for cat_col in claim_df.select_dtypes(include = ['object']).keys().tolist():

    if cat_col != 'EarlyClaimAndLapsed':
        df_g = claim_df[[cat_col, 'EarlyClaimAndLapsed']].groupby([cat_col,
'EarlyClaimAndLapsed']).size().reset_index()
        df_g['percentage'] = claim_df[[cat_col,
'EarlyClaimAndLapsed']].groupby([cat_col,
'EarlyClaimAndLapsed']).size().groupby(level=0).apply(lambda x: 100 * x /
float(x.sum())).values
        df_g.columns = [cat_col, 'EarlyClaimAndLapsed', 'Counts', 'Percentage']
        df_g = df_g.sort_values(['Counts'],
ascending=False).reset_index(drop=True)

        fig = px.bar(df_g, x=cat_col, y=['Counts'],
color='EarlyClaimAndLapsed', text=df_g['Percentage'].apply(lambda x:
'{0:1.2f}%'.format(x)), barmode='group')
        fig.update_layout(title_text=f'{cat_col} vs EarlyClaimAndLapsed',
title_x = 0.5, font_size=12)
        fig.show()
```

```
lapsed_df = claim_df[claim_df['EarlyClaimAndLapsed']=='Y']
non_lapsed_df = claim_df[claim_df['EarlyClaimAndLapsed']=='N']

for con_col in claim_df.select_dtypes(include = ['number']).keys().tolist():

    fig = go.Figure()
    fig.add_trace(go.Histogram(x = non_lapsed_df[con_col], name='N',
marker_color='blue', hovertext=non_lapsed_df['EarlyClaimAndLapsed']))
    fig.add_trace(go.Histogram(x = lapsed_df[con_col], name='Y',
```

```

marker_color='red', hovertext=lapsed_df['EarlyClaimAndLapsed']))
    fig.update_layout(title=f'{con_col} vs EarlyClaimAndLapsed', title_x=0.5,
barmode="overlay",
                        legend_title='EarlyClaimAndLapsed',
xaxis_title=f'{con_col}', yaxis_title='Count'
                        )
    fig.update_traces(opacity=0.5, nbinsx=10, histfunc='count',
hovertemplate="<br>".join([
                        f'{con_col}+": %{x}"',
                        "Count: %{y}",
                        "EarlyClaimAndLapsed: %{hovertext}"
                    ]))
    fig.show()

```

```

for con_col in claim_df.select_dtypes(include = ['number']).keys().tolist():
    fig = px.box(claim_df, y=con_col, color='EarlyClaimAndLapsed')
    fig.update_layout(title_text=f'{con_col} vs EarlyClaimAndLapsed', title_x =
0.5, font_size=12)
    fig.show()

```

```

early = None
not_early = None
target_col = []
cat_cols = []
num_cols = []
bin_cols = []
multi_cols = []
ord_cols = []
nom_cols = []

def update_category_numeric_cols():
    # Separating early and non early customers
    early_fn = claim_df[claim_df['EarlyClaimAndLapsed'] == 'Y']
    not_fn_early = claim_df[claim_df['EarlyClaimAndLapsed'] == 'N']

    # Separating catagorical and numerical columns

```

```

## target features
target_col.clear()
target_col.append('EarlyClaimAndLapsed')

## Category Columns
cat_fn_cols = claim_df.select_dtypes(include = ['object']).keys().tolist()
cat_fn_cols.remove("EarlyClaimAndLapsed")
cat_cols.clear()
for x in cat_fn_cols:
    cat_cols.append(x)

## Numeric columns
num_fn_cols = [x for x in claim_df.columns if x not in cat_cols +
target_col and (claim_df[x].dtypes == np.float64 or claim_df[x].dtypes ==
np.int64)]
num_cols.clear()
for x in num_fn_cols:
    num_cols.append(x)

## Binary columns with 2 values
bin_fn_cols = claim_df.select_dtypes(include =
['object']).nunique()[claim_df.select_dtypes(include = ['object']).nunique() ==
2].keys().tolist()
bin_fn_cols.remove("EarlyClaimAndLapsed")
bin_cols.clear()
for x in bin_fn_cols:
    bin_cols.append(x)

## Columns more than 2 values
multi_fn_cols = [x for x in cat_cols if x not in bin_cols]
multi_cols.clear()
for x in multi_fn_cols:
    multi_cols.append(x)

## Data that have a predetermined/natural order or rank
if 'OCCUPATION_CLASS' in claim_df.columns.tolist() and 'RSK_SUM_ASSURE' in
claim_df.columns.tolist():

```

```

ord_fn_cols = ['OCCUPATION_CLASS', 'RSK_SUM_ASSURE']
elif 'OCCUPATION_CLASS' in claim_df.columns.tolist():
    ord_fn_cols = ['OCCUPATION_CLASS']
elif 'RSK_SUM_ASSURE' in claim_df.columns.tolist():
    ord_fn_cols = ['RSK_SUM_ASSURE']
else:
    ord_fn_cols = []
ord_cols.clear()
for x in ord_fn_cols:
    ord_cols.append(x)

## Data that are classified without ranking
nom_fn_cols = [x for x in cat_cols if x not in ord_cols]
nom_cols.clear()
for x in nom_fn_cols:
    nom_cols.append(x)

## Create a dataframe to view all features type
feature_type = ['Target (Binary Categorical)', 'Categorical (Exclude Target)', 'Numeric', 'Binary Categorical (Exclude Target)', 'Multi-Class Categorical', 'Ordinal Categorical', 'Nominal Categorical']
feature_type = pd.Series(feature_type)

feature_type_count = [len(target_col), len(cat_cols), len(num_cols), len(bin_cols), len(multi_cols), len(ord_cols), len(nom_cols)]
feature_type_count = pd.Series(feature_type_count)

display(Markdown(f"### <u>Overview of Feature Types</u>"))
feature_type_df = pd.concat([feature_type, feature_type_count], axis = 1,
keys = ['Features Type', 'Frequency'])
feature_type_df.sort_values(by='Frequency', ascending=False, inplace=True)
feature_type_df.set_index(keys='Features Type', inplace=True)
return feature_type_df, early_fn, not_fn_early

```

```

feature_type_df, early, not_early = update_category_numeric_cols()
feature_type_df

```

3. Data Preparation

3.1 Selecting Data

Selecting items (rows): In our dataset, there are some outliers which seriously affect the accuracy of our prediction. So, we will remove those outliers in our dataset in order to gain a more accurate and precise prediction when performing modeling. Besides, we will also check whether there are any missing values, and we will take correspondence measures either to remove or replace it with mean values (if any). Please refer to **3.2 Data Cleaning** for further elaboration and explanation.

Selecting attributes (columns): In our data set, there are 52 features (columns), and the last feature called "EarlyClaimAndLapsed" is selected as the outcome. The remaining 51 features in our dataset are selected as the independent variable to predict the outcome. During the data preparation stage, the categorical and continuous data have been identified and both types of data will undergo constant features checking, but an extra checking called highly concentrated features is applied on continuous data. The continuous data will then undergo the data normalisation or standardisation process so that the final value is between 0 and 1.

3.2 Data Cleaning

We usually can't have any missing values in our dataset if we want to use them for predictive modeling. This is because when performing predictive modeling, it may affect accuracy and precision, thereby reducing the reliability and trustworthiness towards the result. Therefore, we will check whether there are any missing values in our dataset first. In our case, there are no missing values in our dataset as we have already removed it before this (**Data Manipulation under 2.2.6**). Besides, we will also check for the occurrence of outliers and remove the outliers using Interquartile Range (IQR).

3.2.1 Remove Missing Values

```
# Check missing values
display(Markdown(f"Total missing values: {claim_df.isnull().sum().sum()}"))
display(Markdown(f"We don't have any missing values in our dataset because we have removed them at an early stage."))
```

Output:

Total missing values: 0

We don't have any missing values in our dataset because we have removed them at an early stage.

3.2.2 Remove Constant & Low Variance Features

```
display(Markdown(f"### Total Rows: {claim_df.shape[0]}"))

mode_df = claim_df[num_cols].mode().iloc[:1, :].T
mode_df.reset_index(inplace=True)
mode_df.rename(columns = {'index': 'Continuous Features', 0: 'Mode Value'},
inplace=True)

concen_df = pd.DataFrame(claim_df[num_cols].var(),
columns=['Variance']).reset_index().rename(columns = {'index': 'Continuous
Features'})
concen_df.drop('Variance', axis=1, inplace=True)
concen_df = pd.merge(concen_df, mode_df, on='Continuous Features')
concen_df['Frequency'] = concen_df['Continuous Features'].apply(lambda x:
claim_df[x].value_counts().tolist()[0])
concen_df.sort_values(by=['Frequency'], inplace=True, ascending=False)
concen_df['Percentage (%)'] = concen_df['Frequency'].apply(lambda x: round(x /
claim_df.shape[0] * 100, 2))
concen_df.reset_index(drop=True, inplace=True)
concen_df
```

```
# Find those continuous features whose mode occurrences occupy 76% or more in
the dataset
concen_df = concen_df.loc[concen_df["Percentage (%)"] >= 76, ['Continuous
Features']]
concen_df
```

```
# Drop above features in our dataset
if len(concen_df.iloc[:, 0].tolist()) > 0:
    claim_df.drop(labels=concen_df.iloc[:, 0].tolist(), axis=1, inplace=True)
    print(f"Successfully drop {concen_df.iloc[:, 0].tolist()}")
else:
    print(f"No low variance numeric features (a.k.a. columns) are needed to
drop.")
```

Output:

No low variance numeric features (a.k.a. columns) are needed to drop.

```

display(Markdown(f"### Total Rows: {claim_df.shape[0]}"))

mode_df = claim_df[cat_cols].mode().iloc[:1, :].T
mode_df.reset_index(inplace=True)
mode_df.rename(columns = {'index': 'Categorical Features', 0: 'Mode Value'},
inplace=True)

# concen_df = pd.DataFrame(claim_df[num_cols].var(),
columns=['Variance']).reset_index().rename(columns = {'index': 'Continuous
Features'})
# concen_df.drop('Variance', axis=1, inplace=True)
# concen_df = pd.merge(concen_df, mode_df, on='Continuous Features')

mode_df['Frequency'] = mode_df['Categorical Features'].apply(lambda x:
claim_df[x].value_counts().tolist()[0])
mode_df.sort_values(by=['Frequency'], inplace=True, ascending=False)
mode_df['Percentage (%)'] = mode_df['Frequency'].apply(lambda x: round(x /
claim_df.shape[0] * 100, 2))
mode_df.reset_index(drop=True, inplace=True)
mode_df

```

```

# Find those category features has only 1 category
mode_df = mode_df.loc[mode_df["Percentage (%)"] >= 95, ['Categorical
Features']]

# Drop those above columns
if len(mode_df.iloc[:, 0].tolist()) > 0:
    claim_df.drop(labels=mode_df.iloc[:, 0].tolist(), axis=1, inplace=True)
    print(f"Successfully drop {mode_df.iloc[:, 0].tolist()}")
else:
    print(f"No low variance categorical features (a.k.a. columns) are needed to
drop.")

```

Output:

Successfully drop ['Type', 'POL HOLDER TYPE']

```

feature_type_df, early, not_early = update_category_numeric_cols()
feature_type_df

```

3.2.3 Remove Outliers

```
# Create Box Plot Function
def vis_box_plot(claim_df):
    dis = 0
    displayable = True
    count_display = 0
    outliers = None
    for index, ele in enumerate(num_cols):
        if (index + 1) % 10 == 0:
            outliers = claim_df[num_cols[int(f'{dis}'):index + 1]].copy()
            dis = index

        elif len(num_cols) < 10 and ele == num_cols[-1]:
            outliers = claim_df[num_cols[int(f'{dis}'):]].copy()

        elif ele == num_cols[-1]:
            outliers = claim_df[num_cols[int(f'{dis+1}'):]].copy()

        else:
            displayable = False

    if displayable:
        fig = px.box(outliers, height=800, width=1000, points="outliers")
        fig.update_layout(title="<b>Outliers Visualisation for All
Continuous Features</b>",
                           yaxis_title='<b>Values</b>',
                           xaxis_title='<b>Continuous Features</b>',
                           width=950, height=800, title_x = 0.5,
                           font_size=12)
        fig.show()
        displayable = True

vis_box_plot(claim_df)
```

```
# Create outliers removal function using Interquartile Range (IQR)
def remove_outliers(claim_df, column_name=None):
```

```

if column_name is not None:
    Q1 = claim_df[column_name].quantile(0.25)
    Q3 = claim_df[column_name].quantile(0.75)
    if Q3 != 0 and str(Q3) != 'nan' and Q3 != Q1:
        IQR = Q3 - Q1
        lower_whisker = Q1 - (1.5 * IQR)
        upper_whisker = Q3 + (1.5 * IQR)
        display(Markdown(f"<u>**{column_name}**</u>"))
        print(f'Q1          : {Q1}\nQ3          : {Q3}\nIQR          :
{IQR}\nlower_whisker: {lower_whisker}\nupper_whisker: {upper_whisker}\n')
        claim_df = claim_df.loc[(claim_df[column_name] > lower_whisker) &
(claim_df[column_name] < upper_whisker)]
    else:
        for col_name, col_type in zip(claim_df.columns.tolist(),
claim_df.dtypes.tolist()):
            if (col_type == 'int64' or col_type == 'float64') and column_name
is None:

                Q1 = claim_df[col_name].quantile(0.25)
                Q3 = claim_df[col_name].quantile(0.75)
                if Q3 != 0 and str(Q3) != 'nan' and Q3 != Q1:
                    IQR = Q3 - Q1
                    lower_whisker = Q1 - (1.5 * IQR)
                    upper_whisker = Q3 + (1.5 * IQR)
                    display(Markdown(f"<u>**{col_name}**</u>"))
                    print(f'Q1          : {Q1}\nQ3          :
{Q3}\nIQR          : {IQR}\nlower_whisker: {lower_whisker}\nupper_whisker:
{upper_whisker}\n')
                    claim_df = claim_df.loc[(claim_df[col_name] >
lower_whisker) & (claim_df[col_name] < upper_whisker)]
            return claim_df

```

Create outliers remaning checking function

```
def check_remaining_outliers(claim_df, num_cols):
```

```
    ## Create a boxplot using matplotlib
```

```
    bp = plt.boxplot(claim_df[num_cols], showmeans=True) # ['SASurgeonExpPay',
```

```
'HSSExpPayTotal', 'HABMedExpPay']
```

```
## Retrieve Q1 value for each feature
```

```
q1 = [round(min(item.get_ydata()), 1) for item in bp['boxes']]
```

```
## Retrieve outliers for each feature
```

```
fliers = [item.get_ydata() for item in bp['fliers']]
```

```
lower_outliers = []
```

```
upper_outliers = []
```

```
for i in range(len(fliers)):
```

```
    lower_outliers_by_box = []
```

```
    upper_outliers_by_box = []
```

```
    for outlier in fliers[i]:
```

```
        if outlier < q1[i]:
```

```
            lower_outliers_by_box.append(round(outlier, 4))
```

```
        else:
```

```
            upper_outliers_by_box.append(round(outlier, 4))
```

```
    lower_outliers_by_box.sort(reverse=True)
```

```
    lower_outliers.append(lower_outliers_by_box)
```

```
    upper_outliers_by_box.sort()
```

```
    upper_outliers.append(upper_outliers_by_box)
```

```
    # print(f'Lower outliers: {lower_outliers}\n'f'Upper outliers:  
{upper_outliers}')
```

```
    return lower_outliers, upper_outliers
```

```
lower_outliers, upper_outliers = check_remaining_outliers(claim_df, num_cols)
```

```
def chkList(lst):
```

```
    if len(lst) >= 8:
```

```
        return True
```

```
    else:
```

```
        return False
```

```
def chkListLength(lst):
```

```
    for index, inner_lst in enumerate(lst):
```

```

        if len(inner_lst) >= 15: # more than 15 outliers data points
            return True
        return False

upper_outliers_col_name =
claim_df.select_dtypes(include=['number']).columns.tolist()

count = 0
while count < 4: # chkListLength(upper_outliers) == True:

    count2 = 0

    while count2 < len(upper_outliers_col_name):

        if chkList(upper_outliers[count2]) == True:
            # Remove outliers
            claim_df = remove_outliers(claim_df,
upper_outliers_col_name[count2])
            count2 += 1

        # Check is there any outliers left
        lower_outliers, upper_outliers = check_remaining_outliers(claim_df,
num_cols)

        display(Markdown(f"### **<u>Loop {count+1} in Removing Outliers</u>**"))
        count += 1

        # Visualise the result
        vis_box_plot(claim_df)

    print()

```

```

feature_type_df, early, not_early = update_category_numeric_cols()
feature_type_df

```

3.2.4 Handling High Cardinality Features

```
# Reset the index of the dataframe so that the index values can be moved out as
the first column in the dataframe.
display(Markdown(f"#### Unique values of each categorical features:"))
cat_unique_df = pd.DataFrame(claim_df[claim_df.select_dtypes(include =
['object']).keys().tolist()].nunique(),
                           columns=['Number of Unique
Values']).reset_index().rename(columns = {'index':'Categorical
Features'}).sort_values(by=['Number of Unique Values'], ascending = False)
cat_unique_df.reset_index(inplace=True, drop=True)
cat_unique_df
```

```
# Select those more than 6 unique categories
cat_unique_df.loc[cat_unique_df['Number of Unique Values'] > 6]
```

```
lst = cat_unique_df.loc[cat_unique_df['Number of Unique Values'] >
6]['Categorical Features'].tolist() + ['EarlyClaimAndLapsed']

sweet_report = sv.analyze(source=claim_df[lst],
target_feat='EarlyClaimAndLapsed')
sweet_report.show_notebook(w="100%", h="Full", layout='vertical')
```

```
# Drop 'SELL_AGENT_OCCUPATION_CODE' and 'SERVICE_AGENT_OCCUPATION_CODE' columns
as
# these columns have too many unique category values

if len(cat_unique_df.loc[cat_unique_df['Number of Unique Values'] >
18]['Categorical Features'].tolist()) != 0:
    claim_df.drop(labels=cat_unique_df.loc[cat_unique_df['Number of Unique
Values'] > 18]['Categorical Features'].tolist(),
                  axis=1, inplace=True) # 'BRANCH_CODE'
    print(f"Successfully drop {cat_unique_df.loc[cat_unique_df['Number of
Unique Values'] > 18]['Categorical Features'].tolist()}")
else:
    print(f"No high cardinality (>18) categorical features (a.k.a. columns) are
needed to drop.")
```

Output:

Successfully drop ['SELL_AGENT_OCCUPATION_CODE', 'SERVICE_AGENT_OCCUPATION_CODE']

```
# Shows the result of 'RISK_CODE' column before reducing the unique features
display(Markdown(f"Before reducing the unique features for `RISK_CODE`
column:**"))
test_df = pd.concat({'Frequencies':
claim_df['RISK_CODE'].value_counts(ascending=True),
                      'Percentage':
claim_df['RISK_CODE'].value_counts(normalize=True, ascending=True)},
                      axis=1
                      ).reset_index().rename(columns = {'index':'Value'})
test_df#.iloc[3:, :].Percentage.sum()
```

```
from collections import Counter
def cumulatively_categorise(column, threshold=0.75,
return_categories_list=True):

    # Find the threshold value using the percentage and number of instances in
the column
    threshold_value=int(threshold * len(column))

    # Initialise an empty list for our new minimised categories
    categories_list = []

    # Initialise a variable to calculate the sum of frequencies
    s = 0

    # Create a counter dictionary of the form unique_value: frequency
    counts = Counter(column)

    #Loop through the category name and its corresponding frequency after
sorting the categories by descending order of frequency
    for i, j in counts.most_common():
        # Add the frequency to the global sum
        s += dict(counts)[i]
        # Append the category name to the list
```

```

        categories_list.append(i)
        # Check if the global sum has reached the threshold value, if so break
the loop
        if s >= threshold_value:
            break

        # Append the category Other to the list
        categories_list.append('Other')

        # Replace all instances not in our new categories by Other
        new_column = column.apply(lambda x: x if x in categories_list else 'Other')

        # Return transformed column and unique values if return_categories=True
        if(return_categories_list):
            return new_column, categories_list
        # Return only the transformed column if return_categories=False
        else:
            return new_column

```

```

high_card_cols = {'STATE': 0.9872200082451559,
                  'SELL_AGENT_STATE': 0.9894187165040538,
                  'RISK_CODE': 0.9891438779716917,
                  'SERVICE_AGENT_STATE': 0.9890064587055105
                  }

for col, thresh in high_card_cols.items():
    transformed_column, new_category_list =
cumulatively_categorise(column=claim_df[col], threshold=thresh,
return_categories_list=True)
    claim_df[col] = transformed_column

```

```

# Shows the result of 'RISK_CODE' column before reducing the unique features
display(Markdown(f"After reducing the unique features for `RISK_CODE`
column:"))
pd.concat({'Frequencies': claim_df['RISK_CODE'].value_counts(ascending=True),
          'Percentage':
claim_df['RISK_CODE'].value_counts(normalize=True, ascending=True)},

```

```
axis=1
).reset_index().rename(columns = {'index':'Value'})
```

3.2.5 Remove Highly Correlated Features

```
cat_target_cols = cat_cols + target_col
corr_df = pd.DataFrame(associations(dataset=claim_df,
numerical_columns=num_cols, nom_nom_assoc='theil', figsize=(20, 20),
nominal_columns=cat_target_cols).get('corr'))
corr_df
```

```
display(Markdown(f"### <u>Correlation of each features against
EarlyClaimAndLapsed:</u>"))
target_corr_dict = dict()
target_corr_lst = list()

EarlyClaimAndLapsed_idx = corr_df.index.tolist().index('EarlyClaimAndLapsed')

for index, col_name in enumerate(corr_df.columns):
    if col_name in claim_df.columns.tolist()[:-1]:
        corr_val = corr_df.loc[corr_df.index.tolist()[EarlyClaimAndLapsed_idx],
col_name] # getting the correlation value
        target_corr_dict[col_name] = corr_val
        target_corr_lst.append([col_name, corr_val])

target_corr_df = pd.DataFrame(target_corr_lst, columns=['Features',
'Correlation Value against EarlyClaimAndLapsed'])
target_corr_df.sort_values(by=['Correlation Value against
EarlyClaimAndLapsed'], ascending=True, inplace=True)
target_corr_df.reset_index(drop=True, inplace=True)
target_corr_df
```

```
display(Markdown(f"### <u>List of features who correlated 80% and above with
each other:</u>"))

row_index = 0
corrDict = {}
```

```

row_name = []
col_name = []
corr_val = []

while row_index < len(corr_df.index.tolist()):
    for index, x in enumerate(corr_df.iloc[row_index, :]):
        if abs(x) >= 0.8 and index != row_index:
            if abs(x) in corr_val:
                if (corr_df.index.tolist()[row_index] in col_name) and
(corr_df.columns.tolist()[index] in row_name):
                    continue
                row_name.append(corr_df.index.tolist()[row_index])
                col_name.append(corr_df.columns.tolist()[index])
                corr_val.append(x)
            row_index += 1

corrDict = {"First Feature (FF)": row_name, "Second Feature (SF)": col_name,
"Correlation (FF x SF)": corr_val}
corr_df2=pd.DataFrame(corrDict)
corr_df2.drop_duplicates(subset=['First Feature (FF)', 'Second Feature (SF)'],
inplace=True, ignore_index=True)
# corr_df2

rows_to_be_dropped = []
looped_lst = []

# Swab first and second column
corr_df3 = corr_df2.iloc[:, [1, 0] + list(range(2, corr_df2.shape[1]))]
corr_df3
for i in range(len(corr_df3)):
    first_two_col_row_data = [corr_df3.iloc[i, 0], corr_df3.iloc[i, 1]]
    for index in corr_df2.index:
        if first_two_col_row_data == corr_df2.iloc[index, 0:2].tolist():
            val1, val2 = first_two_col_row_data
            looped_lst.append([val2, val1])
            if corr_df2.iloc[index, 0:2].tolist() not in looped_lst:
                rows_to_be_dropped.append(index)

```

```
corr_df2.drop(index=rows_to_be_dropped, inplace=True)
corr_df2.reset_index(drop=True, inplace=True)
corr_df2
```

```
# Retrieve all the unique features from above list
features_set = set()
for col_name in corr_df2.columns:
    if type(corr_df2[col_name].tolist()[0]) == str:
        for col in corr_df2[col_name].tolist():
            features_set.add(col)
features_set = list(features_set)
features_set
```

Output:

```
['SELL_AGENT_STATE',
 'SELL_AGENT_AGE',
 'SERVICE_AGENT_STATE',
 'ENTRY_AGE',
 'PAYMNT_TERM',
 'SERVICE_AGENT_AGE']
```

```
target_corr_dict = dict()
target_corr_lst = list()

EarlyClaimAndLapsed_idx = corr_df.index.tolist().index('EarlyClaimAndLapsed')

for index, col_name in enumerate(corr_df.columns):
    if col_name in features_set:
        corr_val = corr_df.loc[corr_df.index.tolist()[EarlyClaimAndLapsed_idx],
col_name] # getting the correlation value
        target_corr_dict[col_name] = corr_val
        target_corr_lst.append([col_name, corr_val])

target_corr_df = pd.DataFrame(target_corr_lst, columns=['Features',
'Correlation Value against EarlyClaimAndLapsed'])
target_corr_df.sort_values(by=['Correlation Value against
EarlyClaimAndLapsed'], ascending=True, inplace=True)
target_corr_df.reset_index(drop=True, inplace=True)
target_corr_df
```

```
display(Markdown(f"### <u>Correlation analysis of features that correlate 80%
and above with each other:</u>"))

corr_df2['Correlation (FF x EarlyClaimAndLapsed)'] = corr_df2['First Feature
(FF)'].map(target_corr_dict)
corr_df2['Correlation (SF x EarlyClaimAndLapsed)'] = corr_df2['Second Feature
(SF)'].map(target_corr_dict)
FFxSFcorr = corr_df2['Correlation (FF x SF)']
corr_df2.drop('Correlation (FF x SF)', axis=1, inplace=True)
corr_df2.insert(loc=corr_df2.shape[1], column='Correlation (FF x SF)',
value=FFxSFcorr)
corr_df2.style.highlight_min(subset=['Correlation (FF x EarlyClaimAndLapsed)',
'Correlation (SF x EarlyClaimAndLapsed)'], axis=1)
```

```
col_set_to_drop = set()
for i in range(len(corr_df2)):
    min_num = min(corr_df2.iloc[i, 2].item(), corr_df2.iloc[i, 3].item())

    if str(corr_df2.iloc[i, 4].item())[0] == '-':
        continue
    elif min_num == corr_df2.iloc[i, 2]:
        col_set_to_drop.add(corr_df2.iloc[i, 0])
    else:
        col_set_to_drop.add(corr_df2.iloc[i, 1])

display(Markdown(f"***Columns to be dropped:***"))
col_set_to_drop
```

Output:

Columns to be dropped:

```
{'SERVICE_AGENT_AGE', 'SERVICE_AGENT_STATE'}
```

```
# Drop all the selected columns above
claim_df.drop(labels=col_set_to_drop, inplace=True, axis=1)
claim_df
```

```
# update category and numeric columns as we have dropped some columns just now
feature_type_df, early, not_early = update_category_numeric_cols()
feature_type_df
```

```
for col in multi_cols:
    for col_name, count in claim_df[col].value_counts().items():
        if count <= 5:
            claim_df = claim_df.loc[(claim_df[col] != col_name)]
            claim_df.reset_index(drop=True, inplace=True)
```

```
# update category and numeric columns as we have dropped some columns just now
feature_type_df, early, not_early = update_category_numeric_cols()
feature_type_df
```

```
# Check duplicate rows again
if claim_df[claim_df.duplicated()].shape[0] > 0:
    print(f"There are {claim_df[claim_df.duplicated()].shape[0]} duplicated rows.")
    print("\nDuplicate rows will be cleared now...")
    claim_df.drop_duplicates(inplace=True)
else:
    print("Congratz! No duplicate rows.")
```

Output:

here are 228 duplicated rows.

Duplicate rows will be cleared now...

```
pie_display = claim_df

# labels
lab = pie_display["EarlyClaimAndLapsed"].value_counts().keys().tolist()

# values
val = claim_df["EarlyClaimAndLapsed"].value_counts().values.tolist()

trace = go.Pie(labels = lab , values = val, marker = dict(colors =
```

```

['blue' , 'green'], line = dict(color = "white", width = 1.3)), rotation = 90,
hoverinfo = "label + value + text", hole = 0.5)
layout = go.Layout(dict(title = "Distribution of Early & Lapsed Claimed in
Data", title_x = 0.5, plot_bgcolor = "rgb(243, 243, 243)", paper_bgcolor =
"rgb(243, 243, 243)",))

import plotly.io as pio
pio.renderers.default='notebook'

fig = go.Figure(data = [trace], layout = layout)
py.iplot(fig)

```

```

# Using sweetwiz to generate report
import warnings
warnings.filterwarnings('ignore')

sweet_report = sv.analyze(source=claim_df, target_feat='EarlyClaimAndLapsed')
sweet_report.show_notebook(w="100%", h="Full", layout='vertical')

```

```

# This function is used to bin numeric features where each category each bin
contain 5 values
# E.g. 0-4, 5-9, and so on
def bin_numeric_cols(val):
    if val < 5:
        return '0-4'
    elif val < 10:
        return '5-9'
    elif val < 15:
        return '10-14'
    elif val < 20:
        return '15-19'
    elif val < 25:
        return '20-24'
    elif val < 30:
        return '25-29'
    elif val < 35:
        return '30-34'

```

```

elif val < 40:
    return '35-39'
elif val < 45:
    return '40-44'
elif val < 50:
    return '45-49'
elif val < 55:
    return '50-54'
elif val < 60:
    return '55-59'
elif val < 65:
    return '60-64'
elif val < 70:
    return '65-69'
elif val < 75:
    return '70-74'
elif val < 80:
    return '75-79'
elif val < 85:
    return '80-84'
elif val < 90:
    return '85-89'
elif val < 95:
    return '90-94'
else:
    return '95-99'

```

```

numBin_df = pd.DataFrame()
for num_col in claim_df.select_dtypes(include = ['number']).keys().tolist():
    if (claim_df.select_dtypes(include=['number'])[num_col].max() < 100) and
(claim_df.select_dtypes(include=['number'])[num_col].min() >= 0):
        numBin_df[f'{num_col}'] = claim_df[num_col].apply(bin_numeric_cols)

numBin_df = numBin_df.merge(claim_df['EarlyClaimAndLapsed'].to_frame(),
left_index=True, right_index=True)
numBin_df

```

```
claim_df.select_dtypes(include =
['number']).merge(claim_df['EarlyClaimAndLapsed'].to_frame(), left_index=True,
right_index=True)
```

```
for cat_col in claim_df.select_dtypes(include = ['object']).keys().tolist():

    if cat_col != 'EarlyClaimAndLapsed':
        df_g = claim_df[[cat_col, 'EarlyClaimAndLapsed']].groupby([cat_col,
'EarlyClaimAndLapsed']).size().reset_index()
        df_g['percentage'] = claim_df[[cat_col,
'EarlyClaimAndLapsed']].groupby([cat_col,
'EarlyClaimAndLapsed']).size().groupby(level=0).apply(lambda x: 100 * x /
float(x.sum())).values
        df_g.columns = [cat_col, 'EarlyClaimAndLapsed', 'Counts', 'Percentage']
        df_g = df_g.sort_values(['Counts'],
ascending=False).reset_index(drop=True)

        fig = px.bar(df_g, x=cat_col, y=['Counts'],
color='EarlyClaimAndLapsed', text=df_g['Percentage'].apply(lambda x:
'{0:1.2f}%'.format(x)), barmode='group')
        fig.update_layout(title_text=f'{cat_col} vs EarlyClaimAndLapsed',
title_x = 0.5, font_size=12)
        fig.update_yaxes(title_text='Count')
        fig.show()
```

```
for cat_col in numBin_df.select_dtypes(include = ['object']).keys().tolist():

    if cat_col != 'EarlyClaimAndLapsed':
        df_g = numBin_df[[cat_col, 'EarlyClaimAndLapsed']].groupby([cat_col,
'EarlyClaimAndLapsed']).size().reset_index()
        df_g['percentage'] = numBin_df[[cat_col,
'EarlyClaimAndLapsed']].groupby([cat_col,
'EarlyClaimAndLapsed']).size().groupby(level=0).apply(lambda x: 100 * x /
float(x.sum())).values
        df_g.columns = [cat_col, 'EarlyClaimAndLapsed', 'Counts', 'Percentage']
        df_g[f'{cat_col}_SORT'] = df_g[cat_col].apply(lambda x: int(x.split('-
```

```

')[-1]))
    df_g = df_g.sort_values(by=[f'{cat_col}_SORT'],
ascending=True).reset_index(drop=True)

    # If the EarlyClaimAndLapsed column contain 'Y' value in the first row
of the dataframe, add a 'N' value to the first row
    if df_g.iloc[0:1, :][f'EarlyClaimAndLapsed'].tolist()[0] == 'Y':
        df_g.loc[-1] = [df_g.iloc[0:1, :][cat_col].tolist()[0], 'N', 0, 0,
df_g.iloc[0:1, :][f'{cat_col}_SORT'].tolist()[0]] # adding a row
        df_g.index = df_g.index + 1 # shifting index
        df_g.sort_index(inplace=True)
        df_g.drop(labels=[f'{cat_col}_SORT'], axis=1, inplace=True)
        # df_g.to_excel(excel_writer=f'df_g_{cat_col}.xlsx', index=False)

    fig = px.bar(df_g, x=cat_col, y=['Counts'],
color='EarlyClaimAndLapsed', text=df_g['Percentage'].apply(lambda x:
'{0:1.2f}%'.format(x)), barmode='group')
    fig.update_layout(title_text=f'{cat_col} vs EarlyClaimAndLapsed',
title_x = 0.5, font_size=12)
    fig.update_yaxes(title_text='Count')
    fig.show()

```

```

lapsed_df = claim_df[claim_df['EarlyClaimAndLapsed']=='Y']
non_lapsed_df = claim_df[claim_df['EarlyClaimAndLapsed']=='N']

for con_col in claim_df.select_dtypes(include = ['number']).keys().tolist():

    fig = go.Figure()
    fig.add_trace(go.Histogram(x = non_lapsed_df[con_col], name='N',
marker_color='blue', hovertext=non_lapsed_df['EarlyClaimAndLapsed']))
    fig.add_trace(go.Histogram(x = lapsed_df[con_col], name='Y',
marker_color='red', hovertext=lapsed_df['EarlyClaimAndLapsed']))
    fig.update_layout(title=f'{con_col} vs EarlyClaimAndLapsed', title_x=0.5,
barmode="overlay",
                    legend_title='EarlyClaimAndLapsed',
xaxis_title=f'{con_col}', yaxis_title='Count'
                    )

```

```

fig.update_traces(opacity=0.5, nbinsx=10, histfunc='count',
hovertemplate="<br>".join([
    f"{con_col}+": "{x}",
    "Count: {y}",
    "EarlyClaimAndLapsed: {hovertext}"
]))
fig.show()

```

```

X = claim_df.drop(labels=['EarlyClaimAndLapsed'], axis=1)
y = claim_df['EarlyClaimAndLapsed']

# Transform categorical features into the appropriate type that is expected by
LightGBM
for c in X.columns:
    col_type = X[c].dtype
    if col_type.name == 'object' or col_type.name == 'category':
        X[c] = X[c].astype('category')

y = y.astype('category')

```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)
y_train = pd.DataFrame(data=y_train)
y_test = pd.DataFrame(data=y_test)

```

```

category_cols = X_train.select_dtypes(include=['category']).columns.to_list()
category_cols

```

Output:

```

['RISK_CODE',
 'SEX',
 'OCCUPATION_CLASS',
 'RACE',
 'MARITAL_STATUS',
 'SMOKER',
 'PAYMENT_MODE',
 'RSK_SUM_ASSURE',
 'EXTRA_LOAD',
 'SERVICE_AGENT_EDU_LEVEL',

```

```
'PAYMENT_METHOD',  
'SELL_AGENT_EDU_LEVEL',  
'SELL_AGENT_STATE',  
'STATE']
```

3.3 Data Transformation (Feature Engineering & Feature Scaling / Encoding)

3.3.1 Feature Encoding (Categorical Data)

```
from category_encoders import OrdinalEncoder  
  
# Show Target Features  
display(Markdown(f"""**<u>{target_col[0]}</u>***"))  
print(f"{claim_df[target_col[0]].unique()}")
```

Output:

```
EarlyClaimAndLapsed  
['Y' 'N']
```

```
# Binary Encoding for target feature which is "EarlyClaimAndLapsed"  
  
maplist = [{'col': 'EarlyClaimAndLapsed', 'mapping': {'Y': 1, 'N': 0}}]  
oe_enc = OrdinalEncoder(mapping=maplist, cols=target_col)  
y_train = oe_enc.fit_transform(y_train)  
y_test = oe_enc.transform(y_test)  
  
y_train
```

3.4 Handle Imbalanced Dataset

Since we are going to build a **tree based model called LightGBM**, we are not required to balance the dataset as **the model can deal with an imbalanced dataset internally** by passing the parameter `is_unbalance=True`. Therefore, we just **print some train test split data shapes details in the cell below**.

```
# Tree-based data
display(Markdown(f"### Tree-based data"))
print('Shape of X: {}'.format(X.shape))
print('Shape of y: {}'.format(y.shape))

print("Number transactions X_train dataset: ", X_train.shape)
print("Number transactions y_train dataset: ", y_train.shape)
print("Number transactions X_test dataset: ", X_test.shape)
print("Number transactions y_test dataset: ", y_test.shape)
```

Output:

```
Tree-based data
Shape of X: (14323, 18)
Shape of y: (14323,)
Number transactions X_train dataset: (11458, 18)
Number transactions y_train dataset: (11458, 1)
Number transactions X_test dataset: (2865, 18)
Number transactions y_test dataset: (2865, 1)
```

4. Modeling

4.1 LightGBM

```
y_train.value_counts()
```

Output:

```
EarlyClaimAndLapsed
0      8220
1      3238
dtype: int64
```

```
from sklearn.utils import class_weight
y_train_class_weights =
class_weight.compute_sample_weight(class_weight='balanced',
y=y_train['EarlyClaimAndLapsed'])
y_train_class_weights
```

Output:

```
array([0.69695864, 0.69695864, 1.76930204, ..., 0.69695864, 1.76930204,
0.69695864])
```

```
pd.Series(y_train_class_weights).value_counts()
```

Output:

```
0.696959    8220
1.769302    3238
dtype: int64
```

```
y_test.value_counts()
```

Output:

```
EarlyClaimAndLapsed
0                    2055
1                    810
dtype: int64
```

```
y_test_class_weights =
class_weight.compute_sample_weight(class_weight='balanced',
y=y_test['EarlyClaimAndLapsed'])
y_test_class_weights
```

Output:

```
array([0.69708029, 1.76851852, 0.69708029, ..., 0.69708029, 0.69708029,
       0.69708029])
```

```
pd.Series(y_test_class_weights).value_counts()
```

Output:

```
0.697080    2055
1.768519    810
dtype: int64
```

Using **is_unbalance** parameter (for dealing imbalanced data) to compute metrics for **10 folds** cross validation.

```
lightgbm_model = LGBMClassifier(boosting_type='gbdt', objective='binary',
random_state=42, is_unbalance=True)

acc_score = cross_val_score(lightgbm_model, X_train, y_train.squeeze(),
scoring='accuracy', cv=10, n_jobs=-1)
precision_score = cross_val_score(lightgbm_model, X_train, y_train.squeeze(),
scoring='precision', cv=10, n_jobs=-1)
f1_score = cross_val_score(lightgbm_model, X_train, y_train.squeeze(),
scoring='f1', cv=10, n_jobs=-1)
```

```

recall_score = cross_val_score(lightgbmc_model, X_train, y_train.squeeze(),
                                scoring='recall', cv=10, n_jobs=-1)
roc_auc_score = cross_val_score(lightgbmc_model, X_train, y_train.squeeze(),
                                scoring='roc_auc', cv=10, n_jobs=-1)

print(f"Accuracy: {acc_score.mean()}")
print(f"Precision: {precision_score.mean()}")
print(f"F1: {f1_score.mean()}")
print(f"Recall: {recall_score.mean()}")
print(f"ROC_AUC: {roc_auc_score.mean()}")

```

Output:

```

Accuracy: 0.6090079791490431
Precision: 0.369883499671485
F1: 0.440758635155494
Recall: 0.5457287008370599
ROC_AUC: 0.6282276484556439

```

Using **scale_pos_weight** parameter (for dealing imbalanced data) to compute metrics for **10 folds** cross validation.

```

lightgbmc_model = LGBMClassifier(boosting_type='gbdt', objective='binary',
                                random_state=42,
                                scale_pos_weight=(y_train.value_counts()[0])/(y_train.value_counts()[1]))

acc_score = cross_val_score(lightgbmc_model, X_train,
                              y_train.squeeze().to_numpy(), scoring='accuracy', cv=10, n_jobs=-1)
precision_score = cross_val_score(lightgbmc_model, X_train,
                                   y_train.squeeze().to_numpy(), scoring='precision', cv=10, n_jobs=-1)
f1_score = cross_val_score(lightgbmc_model, X_train,
                            y_train.squeeze().to_numpy(), scoring='f1', cv=10, n_jobs=-1)
recall_score = cross_val_score(lightgbmc_model, X_train,
                                y_train.squeeze().to_numpy(), scoring='recall', cv=10, n_jobs=-1)
roc_auc_score = cross_val_score(lightgbmc_model, X_train,
                                  y_train.squeeze().to_numpy(), scoring='roc_auc', cv=10, n_jobs=-1)

print(f"Accuracy: {acc_score.mean()}")
print(f"Precision: {precision_score.mean()}")
print(f"F1: {f1_score.mean()}")
print(f"Recall: {recall_score.mean()}")

```

```
print(f"ROC_AUC: {roc_auc_score.mean()}")
```

Output:

Accuracy: 0.6070877249136926
Precision: 0.3682193707491206
F1: 0.4397126369591332
Recall: 0.5463517180751443
ROC_AUC: 0.626489695124883

Scale Pos Weight

```
scale_pos_wgt = (y_train.value_counts()[0])/(y_train.value_counts()[1])  
scale_pos_wgt
```

Output:

2.538604076590488

Hyperparameter Tuning using optuna

```
from optuna.integration import LightGBMPruningCallback  
from sklearn.metrics import accuracy_score, recall_score, precision_score,  
f1_score, roc_curve, roc_auc_score, auc, average_precision_score  
  
def objective(trial, X_train, X_test, y_train, y_test):  
  
    param_grid = {  
        "boosting_type": trial.suggest_categorical('boosting_type', ['gbdt',  
'dart']),  
        "num_leaves": trial.suggest_int("num_leaves", 8, 131072, step=1),  
        "max_depth": trial.suggest_int("max_depth", 3, 48),  
        'n_estimators': trial.suggest_int('n_estimators', 6000, 11000, step=2),  
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3,  
step=0.01),  
        'reg_alpha': trial.suggest_float('reg_alpha', 0.001, 10.0, step=1),  
        'reg_lambda': trial.suggest_float('reg_lambda', 0.001, 10.0, step=1),  
        'min_child_samples': trial.suggest_int('min_child_samples', 5, 200,  
step=1),  
        "min_split_gain": trial.suggest_float("min_split_gain", 0, 15),  
        "subsample": trial.suggest_float("subsample", 0.2, 0.90, step=0.1),  
        "subsample_freq": trial.suggest_categorical("subsample_freq", [1]),  
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.1, 0.90,  
step=0.1),
```

```

        'cat_smooth' : trial.suggest_int('cat_smooth', 10, 100),
        'cat_l2': trial.suggest_int('cat_l2', 1, 20),
        #"pos_subsample": trial.suggest_float("pos_subsample", 0.2, 0.95,
step=0.1),
        #"neg_subsample": trial.suggest_float("neg_subsample", 0.2, 0.95,
step=0.1),
        'random_state': 42,
        'metric': 'auc',
    }

    model = lgbm.LGBMClassifier(objective="binary",
scale_pos_weight=scale_pos_wgt, first_metric_only=True, **param_grid)
    model.fit(
        X=X_train,
        y=y_train.squeeze().to_numpy(),
        sample_weight=y_train_class_weights,
        eval_set=[(X_test, y_test.squeeze().to_numpy())],
        eval_sample_weight=[y_test_class_weights],
        eval_metric="auc",
        eval_sample_weight=[y_test_class_weights],
        early_stopping_rounds=200,
        verbose=-1,
        feature_name=X.columns.tolist(),
        categorical_feature=category_cols,
        callbacks=[LightGBMPruningCallback(trial, "auc")], # Add a pruning
callback
    )
    # y_preds = model.predict(X_test)
    y_pred_proba = model.predict_proba(X_test)[: , 1]

    return roc_auc_score(y_test, y_pred_proba)#
average_precision_score(y_test.squeeze().to_numpy(), y_preds)

```

```

study = optuna.create_study(direction="maximize", study_name="LGBM Classifier")
func = lambda trial: objective(trial, X_train, X_test, y_train, y_test)
study.optimize(func, n_trials=6500, n_jobs=-1)

```

```

print(f'Number of finished trials: {len(study.trials)}')
print()
print(f'Best trial:\n{study.best_trial.params}')
print()
print(f'Best value: {study.best_value}')

```

```

parameters = {}
parameters['random_state'] = 42
parameters['metric'] = 'average_precision'

display(Markdown(f"#### **<u>Best parameters:</u>**"))
for key, value in study.best_params.items():
    print(f"{key}: {value}")
    parameters[key] = value

```

```

display(Markdown(f"#### **<u>Best parameters:</u>**"))
for key, val in parameters.items():
    print(f"{key}: {val}")

```

```

lgbm_model = lgbm.LGBMClassifier(objective="binary",
scale_pos_weight=scale_pos_wgt, **parameters)

lgbm_model.fit(
    X=X_train,
    y=y_train.squeeze().to_numpy(),
    eval_set=[(X_test, y_test)],
    eval_metric="auc",
    early_stopping_rounds=200,
    feature_name=X.columns.tolist(),
    categorical_feature=category_cols,
    verbose=-1
)

```

4.2 LightGBM (Overall Metrics)

4.2.1 Accuracy, Precision, Recall, F1 and AUC Scores

```

y_train_pred = lgbm_model.predict(X_train)
# y_train_pred = [1 if x >= 0.5 else 0 for x in y_train_pred]

y_test_pred = lgbm_model.predict(X_test)
# y_test_pred = [1 if x >= 0.5 else 0 for x in y_test_pred]

print("Accuracy Train: {:.4f}\nAccuracy Test:
{:.4f}".format(accuracy_score(y_train, y_train_pred),
                                                    accuracy_score(y_test,
y_test_pred)))
print()
print("Precision Train: {:.4f}\nPrecision Test:
{:.4f}".format(precision_score(y_train, y_train_pred),
                                                    precision_score(y_test,
y_test_pred)))
print()
print("Recall Train: {:.4f}\nRecall Test: {:.4f}".format(recall_score(y_train,
y_train_pred),
                                                    recall_score(y_test,
y_test_pred)))
print()
print("F1 Train: {:.4f}\nF1 Test: {:.4f}".format(f1_score(y_train,
y_train_pred),
                                                    f1_score(y_test,
y_test_pred)))
print()
print("AUC Train: {:.4f}\nAUC Test: {:.4f}".format(roc_auc_score(y_train,
lgbm_model.predict_proba(X_train)[: ,1]),
                                                    roc_auc_score(y_test,
lgbm_model.predict_proba(X_test)[: ,1])))

```

```

train_score = {}
test_score = {}

for x in [0.5, 0.6, 0.7, 0.8, 0.9]:
    y_train_pred = lgbm_model.predict(X_train)
    # y_test_pred = lgbm_model.predict(X_test)

```

```

y_train_pred = [1 if y >= x else 0 for y in y_train_pred]
# y_test_pred = [1 if y >= x else 0 for y in y_test_pred]

# train
train_score[x] = [accuracy_score(y_train, y_train_pred),
precision_score(y_train, y_train_pred), recall_score(y_train, y_train_pred),
f1_score(y_train, y_train_pred), roc_auc_score(y_train,
lgbm_model.predict_proba(X_train)[: ,1])]

# test
test_score[x] = [accuracy_score(y_test, y_test_pred),
precision_score(y_test, y_test_pred), recall_score(y_test, y_test_pred),
f1_score(y_test, y_test_pred), roc_auc_score(y_test,
lgbm_model.predict_proba(X_test)[: ,1])]

display(Markdown(f'### Train Metrics:'))
train_score = pd.DataFrame(data=train_score, index=['Accuracy', 'Precision',
'Recall', 'F1', 'ROC_AUC']).T
train_score['Threshold'] = train_score.index
first_column = train_score.pop('Threshold')
train_score.insert(0, 'Threshold', first_column)
train_score = train_score.reset_index(drop=True)
train_score

```

```

display(Markdown(f'### Test Metrics:'))
test_score = pd.DataFrame(data=test_score, index=['Accuracy', 'Precision',
'Recall', 'F1', 'ROC_AUC']).T
test_score['Threshold'] = test_score.index

first_column = test_score.pop('Threshold')
test_score.insert(0, 'Threshold', first_column)
test_score = test_score.reset_index(drop=True)
test_score

```

4.2.2 LightGBM (Confusion Matrix)

```

lst = []
for x in [0.5, 0.6, 0.7, 0.8, 0.9]:
    y_test_pred = lgbm_model.predict(X_test)
    # y_test_pred = [1 if y > x else 0 for y in y_test_pred]

    cm = confusion_matrix(y_test, y_test_pred)

    group_names = ['True Neg', 'False Pos', 'False Neg', 'True Pos']
    group_counts = ['{0:0.0f}'.format(value) for value in cm.flatten()]
    group_percentages = ['{0:.2%}'.format(value) for value in
cm.flatten()/np.sum(cm)]
    labels = [f"{v1}\n{v2}\n{v3}" for v1, v2, v3 in zip(group_names,
group_counts, group_percentages)]
    labels = np.asarray(labels).reshape(2,2)
    lst.append((cm, labels))

fig, axes = plt.subplots(2, 3, figsize=(16, 9))
fig.suptitle('Confusion Matrix by Threshold', fontsize=15)

sns.heatmap(ax=axes[0, 0], data=lst[0][0], xticklabels = ['Predicted_No',
'Predicted_Yes'], yticklabels = ['Actual_No', 'Actual_Yes'], annot = lst[0][1],
fmt = '', annot_kws = {'fontsize':20}, cmap="YlGnBu", square = True)
axes[0, 0].set_title("LightGBM Classifier (Thresh: 0.5)")

sns.heatmap(ax=axes[0, 1], data=lst[1][0], xticklabels = ['Predicted_No',
'Predicted_Yes'], yticklabels = ['Actual_No', 'Actual_Yes'], annot = lst[1][1],
fmt = '', annot_kws = {'fontsize':20}, cmap="YlGnBu", square = True)
axes[0, 1].set_title("LightGBM Classifier (Thresh: 0.6)")

sns.heatmap(ax=axes[0, 2], data=lst[2][0], xticklabels = ['Predicted_No',
'Predicted_Yes'], yticklabels = ['Actual_No', 'Actual_Yes'], annot = lst[2][1],
fmt = '', annot_kws = {'fontsize':20}, cmap="YlGnBu", square = True)
axes[0, 2].set_title("LightGBM Classifier (Thresh: 0.7)")

sns.heatmap(ax=axes[1, 0], data=lst[3][0], xticklabels = ['Predicted_No',
'Predicted_Yes'], yticklabels = ['Actual_No', 'Actual_Yes'], annot = lst[3][1],
fmt = '', annot_kws = {'fontsize':20}, cmap="YlGnBu", square = True)
axes[1, 0].set_title("LightGBM Classifier (Thresh: 0.8)")

```

```
sns.heatmap(ax=axes[1, 1], data=lst[4][0], xticklabels = ['Predicted_No',
'Predicted_Yes'], yticklabels = ['Actual_No', 'Actual_Yes'], annot = lst[4][1],
fmt = '', annot_kws = {'fontsize':20}, cmap="YlGnBu", square = True)
axes[1, 1].set_title("LightGBM Classifier (Thresh: 0.9)")
```

```
for x in [0.5, 0.6, 0.7, 0.8, 0.9]:
    y_test_pred = lgbm_model.predict(X_test)
    # y_test_pred = [1 if y > x else 0 for y in y_test_pred]

    cm = confusion_matrix(y_test, y_test_pred)

    group_names = ['True Neg', 'False Pos', 'False Neg', 'True Pos']
    group_counts = ['{0:0.0f}'.format(value) for value in cm.flatten()]
    group_percentages = ['{0:.2%}'.format(value) for value in
cm.flatten()/np.sum(cm)]
    labels = [f"{v1}\n{v2}\n{v3}" for v1, v2, v3 in zip(group_names,
group_counts, group_percentages)]
    labels = np.asarray(labels).reshape(2,2)

    plt.figure(figsize = (6,6))
    sns.heatmap(cm, xticklabels = ['Predicted_No', 'Predicted_Yes'],
yticklabels = ['Actual_No', 'Actual_Yes'],
    annot = labels, fmt = '', annot_kws = {'fontsize':20}, cmap="YlGnBu",
square = True)
    plt.title(f"LightGBM Classifier (Thresh: {x})")
    plt.subplots_adjust(wspace = .3,hspace = .3,)
```

4.2.3 LightGBM (ROC AUC Curve)

```
def Calculate_Plot_ROC(y_test, y_pred, name):
    # Calculate ROC curve
    fpr, tpr, thr = roc_curve(y_test, y_pred)
    roc_auc = auc(fpr,tpr)

    # Plot the ROC curve
    plt.plot([0, 1], [0, 1], 'k--', label = 'Random')
```

```
plt.plot(fpr, tpr, label = 'ROC Curve (Area = %0.2f)' %roc_auc)
plt.xlabel('False positive rate')
plt.ylabel('True positive rate')
plt.title(f'ROC Curve for {name}')
plt.grid()
plt.legend(loc = 'best')
```

```
Calculate_Plot_ROC(y_test, lgbm_model.predict_proba(X_test)[:, 1], 'LightGBM Classifier')
```

4.2.4 LightGBM (Feature Importance)

```
lgbm.plot_importance(booster=lgbm_model, max_num_features=X_train.shape[1])

df_feature_importance = (
    pd.DataFrame({
        'feature': lgbm_model.feature_name_,
        'importance': lgbm_model.feature_importances_,
    })
    .sort_values('importance', ascending=False)
)
df_feature_importance
```

```
explainer = shap.TreeExplainer(lgbm_model)
shap_values = explainer.shap_values(X)
shap.summary_plot(shap_values, X)
```

4.2.5 LightGBM (Interpretation)

```
for name in X_train.columns:
    try:
        shap.dependence_plot(ind=name, shap_values=shap_values[1], features=X,
        feature_names=X_train.columns.tolist(), display_features=X)
    except (TypeError, ValueError):
        continue
```

4.2.6 LightGBM (Save Model)

```
# Save model
import joblib
joblib.dump(lgbm_model, 'lgbm_model_avg_precision.pkl')
```

4.2.7 LightGBM (Load Model)

```
# Load model
import joblib
lgbm_model = joblib.load('lgbm_model_auc.pkl')
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)
y_train = pd.DataFrame(data=y_train)
y_test = pd.DataFrame(data=y_test)
```

```
category_cols = X_train.select_dtypes(include=[ 'category' ]).columns.to_list()
category_cols
```

Output:

```
['RISK_CODE ',
 'SEX',
 'OCCUPATION_CLASS',
 'RACE ',
 'MARITAL_STATUS ',
 'SMOKER',
 'PAYMENT_MODE',
 'RSK_SUM_ASSURE ',
 'EXTRA_LOAD',
 'SERVICE_AGENT_EDU_LEVEL ',
 'PAYMENT_METHOD',
 'SELL_AGENT_EDU_LEVEL ',
 'SELL_AGENT_STATE',
 'STATE']
```

```
from category_encoders import OrdinalEncoder

# Binary Encoding for target feature which is "EarlyClaimAndLapsed"
maplist = [{'col': 'EarlyClaimAndLapsed', 'mapping': {'Y': 1, 'N': 0}}]
oe_enc = OrdinalEncoder(mapping=maplist, cols=target_col)
y_train = oe_enc.fit_transform(y_train)
```

```
y_test = oe_enc.transform(y_test)
```

```
y_train
```

```
from sklearn.metrics import accuracy_score, recall_score, precision_score,
f1_score, roc_curve, roc_auc_score, auc
y_train_pred = lgbm_model.predict(X_train)
y_test_pred = lgbm_model.predict(X_test)

print("Accuracy Train: {:.4f}\nAccuracy Test:
{:.4f}".format(accuracy_score(y_train, y_train_pred),
                                accuracy_score(y_test,
y_test_pred)))
print()
print("Precision Train: {:.4f}\nPrecision Test:
{:.4f}".format(precision_score(y_train, y_train_pred),
                                precision_score(y_test,
y_test_pred)))
print()
print("Recall Train: {:.4f}\nRecall Test: {:.4f}".format(recall_score(y_train,
y_train_pred),
                                recall_score(y_test,
y_test_pred)))
print()
print("F1 Train: {:.4f}\nF1 Test: {:.4f}".format(f1_score(y_train,
y_train_pred),
                                f1_score(y_test,
y_test_pred)))
print()
print("AUC Train: {:.4f}\nAUC Test: {:.4f}".format(roc_auc_score(y_train,
lgbm_model.predict_proba(X_train)[: ,1]),
                                roc_auc_score(y_test,
lgbm_model.predict_proba(X_test)[: ,1]))))
```

Output:

Accuracy Train: 0.9111

Accuracy Test: 0.9065

Precision Train: 0.8124

Precision Test: 0.7927

Recall Train: 0.8910

Recall Test: 0.9062

F1 Train: 0.8499

F1 Test: 0.8456

AUC Train: 0.9561

AUC Test: 0.9591

```
train_score = {}
test_score = {}

for x in [0.5, 0.6, 0.7, 0.8, 0.9]:
    y_train_pred = lgbm_model.predict(X_train)
    # y_test_pred = lgbm_model.predict(X_test)

    y_train_pred = [1 if y >= x else 0 for y in y_train_pred]
    # y_test_pred = [1 if y >= x else 0 for y in y_test_pred]

    # train
    train_score[x] = [accuracy_score(y_train, y_train_pred),
precision_score(y_train, y_train_pred), recall_score(y_train, y_train_pred),
f1_score(y_train, y_train_pred), roc_auc_score(y_train,
lgbm_model.predict_proba(X_train)[: ,1])]

    # test
    test_score[x] = [accuracy_score(y_test, y_test_pred),
precision_score(y_test, y_test_pred), recall_score(y_test, y_test_pred),
f1_score(y_test, y_test_pred), roc_auc_score(y_test,
lgbm_model.predict_proba(X_test)[: ,1])]

display(Markdown(f'### Train Metrics:'))
train_score = pd.DataFrame(data=train_score, index=['Accuracy', 'Precision',
'Recall', 'F1', 'ROC_AUC']).T
train_score['Threshold'] = train_score.index
first_column = train_score.pop('Threshold')
train_score.insert(0, 'Threshold', first_column)
```

```
train_score = train_score.reset_index(drop=True)
train_score
```

```
display(Markdown(f'### Test Metrics:'))
test_score = pd.DataFrame(data=test_score, index=['Accuracy', 'Precision',
'Recall', 'F1', 'ROC_AUC']).T
test_score['Threshold'] = test_score.index

first_column = test_score.pop('Threshold')
test_score.insert(0, 'Threshold', first_column)
test_score = test_score.reset_index(drop=True)
test_score
```

```
y_pred = lgbm_model.predict(X_test)

cm = confusion_matrix(y_test, y_pred)

group_names = ['True Neg', 'False Pos', 'False Neg', 'True Pos']
group_counts = ['{0:0.0f}'.format(value) for value in cm.flatten()]
group_percentages = ['{0:.2%}'.format(value) for value in
cm.flatten()/np.sum(cm)]
labels = [f"{v1}\n{v2}\n{v3}" for v1, v2, v3 in zip(group_names, group_counts,
group_percentages)]
labels = np.asarray(labels).reshape(2,2)

plt.figure(figsize = (6,6))
sns.heatmap(cm, xticklabels = ['Predicted_No', 'Predicted_Yes'], yticklabels =
['Actual_No', 'Actual_Yes'],
annot = labels, fmt = '', annot_kws = {'fontsize':20}, cmap="YlGnBu", square =
True)
plt.title(f"LightGBM Classifier")
plt.subplots_adjust(wspace = .3,hspace = .3,)
```

```
def Calculate_Plot_ROC(y_test, y_pred, name):
    # Calculate ROC curve
    fpr, tpr, thr = roc_curve(y_test, y_pred)
```

```

roc_auc = auc(fpr,tpr)

# Plot the ROC curve
plt.plot([0, 1], [0, 1], 'k--', label = 'Random')
plt.plot(fpr, tpr, label = 'ROC Curve (Area = %0.2f)' %roc_auc)
plt.xlabel('False positive rate')
plt.ylabel('True positive rate')
plt.title(f'ROC Curve for {name}')
plt.grid()
plt.legend(loc = 'best')

```

```

Calculate_Plot_ROC(y_test, lgbm_model.predict_proba(X_test)[:, 1], 'LightGBM
Classifier')

```

```

lgbm.plot_importance(booster=lgbm_model.booster_,
max_num_features=X_train.shape[1])

df_feature_importance = (
    pd.DataFrame({
        'feature': lgbm_model.feature_name_,
        'importance': lgbm_model.feature_importances_,
    })
    .sort_values('importance', ascending=False)
)
df_feature_importance

```

```

lgbm.plot_metric(booster=lgbm_model.eval_result_)

```

```

explainer = shap.TreeExplainer(lgbm_model)
shap_values = explainer.shap_values(X)
shap.summary_plot(shap_values, X)

```

```

for col_name in X_train.columns:
    try:
        shap.dependence_plot(ind=col_name, shap_values=shap_values[1],

```

```
features=X, feature_names=X_train.columns.tolist(), display_features=X)
    except (TypeError, ValueError):
        continue
```